

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Бизнес-проект "Информационно-аналитическая система для РИО вуза.

Разработка сервиса "Методические указания"

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

М.Е. Петров

(инициалы, фамилия)

Группа ПО-226

Руководитель ВКР

(подпись, дата)

Е. И. Аникина

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

«____» _____ 20____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Петров М.Е., шифр 22-06-0019, группа ПО-226

1. Тема «Бизнес-проект "Информационно-аналитическая система для РИО вуза. Разработка сервиса "Методические указания"» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1585-с.

2. Срок предоставления работы к защите «09» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) Провести анализ деятельности редакционно-издательского отдела вуза и выявить проблематику традиционного подхода к проверке учебно-методических материалов;

2) Разработать концептуальную модель информационно-аналитической системы;

3) Спроектировать архитектуру системы, включая базу данных и веб-интерфейс;

4) Реализовать систему средствами веб-технологий (Python, FastAPI, PostgreSQL, HTML, CSS, JavaScript);

5) Разработать модуль автоматической проверки оформления документов формата DOCX (шрифт, отступы, интервалы, нумерация страниц);

6) Разработать модуль генерации титульных листов для методических указаний, учебных пособий и монографий.

3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для программной системы являются: файлы формата DOCX (методические указания, учебные пособия, монографии), параметры для генерации титульных листов (название, автор, рецензенты, УДК, ISBN, описание), учётные данные пользователей (ФИО, email, пароль, факультет, кафедра, роль).

2) Выходными данными являются: результаты автоматической проверки документа (список ошибок с указанием страниц), исправленный документ с визуально выделенными ошибками, PDF-отчёт о проверке, сгенерированный титульный лист в формате DOCX, статистические отчёты по загруженным документам, JWT-токены для авторизации, подтверждение отправки файлов на электронную почту РИО.

4. Содержание работы (по разделам):

4.1. Введение.

4.2. Анализ предметной области.

4.3. Техническое задание: основание для разработки, назначение разработки, требования к программной системе, требования к оформлению документации.

4.4. Технический проект: общие сведения о программной системе, проект данных программной системы (ER-модель, реляционная модель), проектирование архитектуры программной системы, проектирование пользовательского интерфейса программной системы.

4.5. Рабочий проект: маршруты приложения, спецификация контроллеров, системное тестирование разработанного веб-сайта.

4.6. Заключение.

4.7. Список использованных источников.

5. Перечень графического материала:

Лист 1. Сведения о ВКРБ.

Лист 2. Цели и задачи разработки.

Лист 3. ER-диаграмма базы данных.

Лист 4. Реляционная модель базы данных.

Лист 5. Страница генерации титульных листов.

Лист 6. Страница отправки в РИО.

Лист 7. Страница админ панели.

Лист 8. Заключение.

Руководитель ВКР

(подпись, дата)

Е. И. Аникина

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

М.Е. Петров

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 105 страницам. Работа содержит 11 иллюстраций, 14 таблиц и 54 библиографических источников. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: Python, REST API, FastAPI, PostgreSQL, информационно-аналитическая система, проверка учебно-методических материалов, веб-приложение, генерация титульных листов, PDF-отчёты.

Объектом разработки является информационно-аналитическая система автоматизации проверки и обработки учебно-методических материалов редакционно-издательского отдела.

Целью работы является разработка информационно-аналитической системы для автоматизации процессов проверки, оформления и сопровождения учебно-методических документов.

Разработанная система реализована на языке Python с использованием web-фреймворка FastAPI и СУБД PostgreSQL. Клиентская часть разработана с применением HTML, CSS и JavaScript. Взаимодействие между компонентами системы осуществляется через REST API.

Архитектура системы построена по клиент-серверной модели. Серверная часть выполняет обработку запросов пользователей, проверку DOCX-документов, генерацию титульных листов, формирование PDF-отчётов и взаимодействие с базой данных. Клиентская часть предоставляет web-интерфейс для загрузки документов и просмотра результатов проверки.

В системе реализован модуль автоматической проверки DOCX-документов, выполняющий анализ параметров форматирования и структуры учебно-методических материалов. Результаты проверки используются для формирования PDF-отчётов с перечнем найденных ошибок..

Разработанная система позволяет сократить время проверки документов, уменьшить количество ошибок оформления и повысить эффективность работы редакционно-издательского отдела.

ABSTRACT

The volume of work is 105 pages. The work contains 11 illustrations, 14 tables and 54 bibliographic sources. The number of applications is 2. The graphic material is presented in Appendix A. Fragments of the source code are presented in Appendix B.

List of keywords: Python, REST API, FastAPI, PostgreSQL, information and analytical system, verification of teaching materials, web application, generation of cover pages, PDF reports.

The object of the development is an information and analytical automation system for checking and processing educational and methodological materials of the editorial and publishing department.

The purpose of the work is to develop an information and analytical system for automating the processes of verification, registration and maintenance of educational and methodological documents.

The developed system is implemented in Python using the FastAPI web framework and the PostgreSQL database management system. The client side is developed using HTML, CSS, and JavaScript. The interaction between the system components is carried out through the REST API.

The architecture of the system is based on the client-server model. The server side processes user requests, validates DOCX documents, generates title pages, generates PDF reports, and interacts with the database. The client side provides a web interface for downloading documents and viewing the verification results.

The system implements an automatic verification module for DOCX documents, which analyzes the formatting parameters and structure of teaching materials. The verification results are used to generate PDF reports with a list of errors found..

The developed system makes it possible to reduce the time needed to check documents, reduce the number of design errors, and improve the efficiency of the editorial and publishing department.

СОДЕРЖАНИЕ

РЕЗЮМЕ СТАРТАП-ПРОЕКТА	12
ВВЕДЕНИЕ	14
1 Анализ предметной области	18
1.1 Характеристика деятельности редакционно-издательского отдела вуза	18
1.2 Проблемы традиционного подхода к подготовке к изданию мето- дических материалов	19
1.3 Обоснование необходимости автоматизации	20
1.4 Анализ существующих систем автоматизации документооборота и проверки документов	21
1.4.1 Microsoft Word и встроенные средства рецензирования	21
1.4.2 Google Docs	21
1.4.3 Системы электронного документооборота	22
1.4.4 Вывод по анализу аналогов	22
1.5 Постановка задач на разработку	23
2 Техническое задание	24
2.1 Основание для разработки	24
2.2 Цель и назначение разработки	24
2.3 Требования к функциональности	24
2.4 Требования к интерфейсу	25
2.5 Требования к обработке ошибок	29
2.6 Требования к удобству использования	30
2.7 Моделирование вариантов использования	30
2.7.1 Вариант использования «Вход в систему»	33
2.7.2 Вариант использования «Выход из системы»	34
2.7.3 Вариант использования «Загрузка документа на проверку»	34
2.7.4 Вариант использования «Автоматическая проверка документа»	35
2.7.5 Вариант использования «Получение отчёта об ошибках»	35

2.7.6	Вариант использования «Скачивание документа с выделенными ошибками»	36
2.7.7	Вариант использования «Автоматическое сохранение корректного документа»	36
2.7.8	Вариант использования «Просмотр архива документов»	37
2.7.9	Вариант использования «Формирование титульного листа»	37
2.7.10	Вариант использования «Просмотр статистики»	38
2.7.11	Вариант использования «Управление пользователями»	38
2.7.12	Вариант использования «Настройка параметров системы»	38
2.8	Требования к надёжности	39
2.9	Требования к информационной безопасности	40
2.10	Требования к программной и технической совместимости	40
2.11	Требования к оформлению документации	41
3	Технический проект	42
3.1	Архитектура программной системы	42
3.2	Модуль обработки учебно-методических документов	44
3.2.1	Общая архитектура модуля	44
3.2.2	Интеграция с Microsoft Word через COM-интерфейс	44
3.2.3	Алгоритм автоматической проверки оформления	45
3.2.4	Формирование исправленной копии документа	49
3.2.5	Генерация PDF-отчёта	49
3.2.6	Генерация титульных листов	50
3.2.7	Управление сессиями и обновление токенов	51
3.2.8	Взаимодействие с базой данных через ORM	52
3.2.9	Предотвращение дублирования	53
3.2.10	Обработка ошибок и восстановление	54
3.2.11	Хранение данных	54
3.2.12	Логирование	55
3.2.13	Заключение	55
3.3	Проект базы данных	56
3.3.1	Схема базы данных	56

3.3.2	Нормализация	58
3.3.3	Реляционная модель данных	58
3.3.4	Описание структуры базы данных	60
4	Рабочий проект	63
4.1	Спецификация компонентов программной системы	63
4.1.1	Класс WordMethodicalChecker (модуль checker.py)	63
4.1.2	Класс CheckReport (модуль checker.py)	64
4.1.3	Модуль генерации титульных листов (titlePageGenerator.py)	65
4.1.4	Модуль формирования PDF-отчётов (pdfReport.py)	66
4.1.5	Модуль отправки в РИО (email.py)	67
4.1.6	Контроллеры REST API (routers)	67
4.2	Системное тестирование	69
4.2.1	Сводная таблица тест-кейсов	69
4.2.2	ТС-01. Проверка корректного DOCX-документа	70
4.2.3	ТС-02. Проверка документа с ошибками оформления	70
4.2.4	ТС-03. Генерация титульного листа методических указаний	71
4.2.5	ТС-04. Генерация титульного листа учебного пособия	71
4.2.6	ТС-05. Генерация титульного листа монографии	71
4.2.7	ТС-06. Отправка документов в РИО (полный цикл)	72
4.2.8	ТС-07. Отправка в РИО с ошибкой на шаге 2	72
4.2.9	ТС-08. Скачивание исправленного документа	73
4.2.10	ТС-09. Скачивание PDF-отчёта	73
4.2.11	Нефункциональное тестирование	73
4.2.12	Результаты тестирования	74
	ЗАКЛЮЧЕНИЕ	75
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	76
	ПРИЛОЖЕНИЕ А Представление графического материала	82
	ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	91
	На отдельных листах (CD-RW в прикрепленном конверте)	105
	Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
	Цели и задачи разработки (Графический материал / Цели и задачи разработ-	

ки.png)	Лист 2
ER-диаграмма базы данных (Графический материал / ER-диаграмма базы данных.png)	Лист 3
Реляционная модель базы данных (Графический материал / Реляционная модель базы данных.png)	Лист 4
Страница генерации титульных листов (Графический материал / Страница генерации титульных листов.png)	Лист 5
Страница отправки в РИО (Графический материал / Страница отправки в РИО.png)	Лист 6
Страница админ панели (Графический материал / Страница админ панели.png)	Лист 7
Заключение (Графический материал / Заключение.png)	Лист 8

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

РИО — редакционно-издательский отдел.

СУБД — система управления базами данных.

ТЗ — техническое задание.

ТП — технический проект.

ФИО — фамилия, имя, отчество.

API (Application Programming Interface) — программный интерфейс приложения.

CSS (Cascading Style Sheets) — каскадные таблицы стилей.

DOCX — формат файлов текстового редактора Microsoft Word, основанный на Office Open XML.

HTML (HyperText Markup Language) — язык разметки гипертекста.

HTTP (HyperText Transfer Protocol) — протокол передачи гипертекста.

HTTPS (HyperText Transfer Protocol Secure) — защищённый протокол передачи гипертекста.

JSON (JavaScript Object Notation) — текстовый формат обмена данными на основе синтаксиса JavaScript.

PDF (Portable Document Format) — переносимый формат документов.

REST (Representational State Transfer) — архитектурный стиль взаимодействия компонентов распределённой системы.

SPA (Single Page Application) — одностраничное веб-приложение.

SQL (Structured Query Language) — язык структурированных запросов.

XSS (Cross-Site Scripting) — межсайтовый скриптинг.

РЕЗЮМЕ СТАРТАП-ПРОЕКТА

Название: «Информационно-аналитическая система автоматизации проверки и оформления учебно-методических материалов редакционно-издательского отдела». Проект представляет собой web-ориентированную систему, предназначенную для автоматизации процессов проверки, оформления и обработки учебно-методических документов в редакционно-издательском отделе образовательной организации.

Система обеспечивает автоматическую проверку DOCX-документов на соответствие требованиям оформления, генерацию титульных листов, формирование PDF-отчётов о найденных ошибках, а также централизованное хранение и обработку результатов проверки.

Ключевые возможности системы включают:

1. Автоматическую проверку учебно-методических материалов на соответствие установленным требованиям оформления с выделением найденных ошибок.
2. Генерацию титульных листов для методических указаний, учебных пособий и монографий на основе пользовательских данных.
3. Формирование PDF-отчётов с результатами проверки документов и статистикой найденных нарушений.
4. Реализацию системы аутентификации и разграничения прав доступа пользователей.
5. Ведение статистики проверок и централизованное хранение информации о документах и пользователях.
6. Интуитивно понятный web-интерфейс, обеспечивающий удобную работу пользователей с системой.

Разработанная система позволяет существенно сократить время проверки учебно-методических материалов, уменьшить количество ошибок оформления и повысить эффективность работы редакционно-издательского отдела.

Актуальность проекта обусловлена необходимостью автоматизации процессов проверки и оформления документов в образовательных организациях. Традиционная ручная проверка требует значительных временных затрат и сопровождается высокой вероятностью появления ошибок, связанных с человеческим фактором.

Целью разработки является создание информационно-аналитической системы, обеспечивающей:

- автоматизированную проверку учебно-методических материалов;
- автоматическое формирование титульных листов и сопроводительной документации;
- повышение точности проверки документов;
- сокращение времени обработки учебных изданий;
- централизованное управление процессами проверки и хранения документов.

Отличительные особенности разработанной системы по сравнению с существующими решениями:

1. Комплексная автоматизация процессов проверки и оформления учебно-методических материалов в рамках единой системы.
2. Автоматическое формирование PDF-отчётов с визуализацией найденных ошибок.
3. Поддержка генерации титульных листов для различных типов учебных изданий.
4. Использование клиент-серверной архитектуры, обеспечивающей возможность масштабирования системы.
5. Гибкая система управления пользователями и разграничения прав доступа.

В системе реализованы функции автоматической проверки DOCX-документов, генерации титульных листов, формирования PDF-отчётов, ведения статистики проверок, а также механизмы авторизации и взаимодействия клиентской и серверной части через REST API.

Идея создания проекта возникла в 2025 году в связи с необходимостью автоматизации процессов проверки учебно-методических материалов в редакционно-издательском отделе образовательной организации. Разработка системы была начата в 2026 году в рамках выполнения выпускной квалификационной работы.

Бизнес-цели:

1. Снижение временных затрат на проверку и оформление учебно-методических материалов.
2. Повышение качества и единообразия оформления документов.
3. Сокращение количества ошибок, связанных с ручной проверкой документов.
4. Повышение эффективности взаимодействия сотрудников редакционно-издательского отдела и преподавателей.
5. Возможность дальнейшего расширения функциональности системы и интеграции с внутренними информационными системами образовательной организации.

ВВЕДЕНИЕ

Вузы постоянно наращивают объемы учебно-методической документации, поэтому автоматизация нормоконтроля и оформления текстов становится критически важной. Сейчас сотрудники редакционно-издательских отделов (РИО) тратят много рабочих часов на то, чтобы вручную сверять учебные пособия и методички с требованиями стандартов. Рутинная проверка не только отнимает время, но и неизбежно ведет к человеческим ошибкам, снижая итоговое качество выпускаемых материалов.

Переход к веб-ориентированным системам помогает решить эту проблему. Подобные информационно-аналитические платформы берут на себя первичный анализ текстов, самостоятельно генерируют отчеты об ошибках и собирают титульные листы. За счет этого авторы методичек и проверяющие из РИО могут взаимодействовать гораздо быстрее в едином цифровом пространстве.

Разработанная в рамках диплома система берет на себя автоматическую проверку учебных материалов на соответствие стандартам оформления. На практике алгоритм работает так: автор загружает файл и получает обратно документ, где все найденные нарушения подсвечены визуально, а также может скачать подробный отчет. Дополнительно встроен модуль формирования титульных листов по утвержденным ГОСТам. В свою очередь, работники РИО получают базу проверенных документов и удобный инструмент для сбора статистики.

Цель настоящей работы – спроектировать и реализовать информационно-аналитическую систему, которая автоматизирует проверку и обработку учебно-методической документации в РИО.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Исследовать специфику работы редакционно-издательского отдела и выявить узкие места в ручной проверке документов.

2. Провести обзор готовых программных решений для автоматизации документооборота и нормоконтроля.
3. Спроектировать архитектуру будущей информационно-аналитической системы.
4. Реализовать серверную часть с использованием FastAPI и PostgreSQL.
5. Разработать клиентскую часть веб-приложения.
6. Реализовать механизмы проверки документов и анализа параметров форматирования.
7. Разработать модули генерации титульных листов и формирования PDF-отчётов.
8. Провести тестирование разработанной системы и оценить корректность её работы.

Структура и объём работы. Отчёт состоит из введения, четырёх разделов основной части, заключения, списка использованных источников и одного приложения. Текст выпускной квалификационной работы равен 105 страницам.

Во введении сформулирована цель работы, поставлены задачи разработки, описана структура работы, приведено краткое содержание каждого из разделов.

В первом разделе на стадии анализа предметной области приводится характеристика деятельности редакционно-издательского отдела, выявляются проблемы традиционного подхода к проверке материалов, обосновывается необходимость автоматизации, анализируются существующие системы и формулируются задачи на разработку.

Во втором разделе на стадии технического задания приводятся требования к разрабатываемой системе: функциональные требования, требования к интерфейсу, к обработке ошибок, к удобству использования, к надёжности, информационной безопасности и совместимости. Также представлены модели вариантов использования.

В третьем разделе на стадии технического проектирования представлены архитектура программной системы, описание модуля обработки учебно-методических документов, проект базы данных (ER-модель, нормализация, реляционная модель).

В четвёртом разделе на стадии рабочего проекта приводится спецификация компонентов клиентской части, описание функций и модулей JavaScript, результаты тестирования модулей для проверки файлов.

В заключении излагаются основные результаты работы, полученные в ходе разработки.

В приложении А представлен графический материал (рисунки, диаграммы).

1 Анализ предметной области

1.1 Характеристика деятельности редакционно-издательского отдела вуза

Редакционно-издательский отдел (РИО) высшего учебного заведения является структурным подразделением, обеспечивающим подготовку, проверку, оформление и выпуск учебно-методической, научной и справочной литературы, используемой в образовательном процессе. К числу основных материалов, поступающих в РИО, относятся методические указания, учебные пособия, практикумы, монографии, сборники научных трудов и иные издания.

Современный образовательный процесс требует постоянного обновления учебно-методической базы. Изменение федеральных государственных образовательных стандартов, развитие научных направлений, внедрение новых технологий обучения и цифровых платформ обуславливают регулярную подготовку новых материалов преподавателями университета.

РИО выполняет функции организационного и нормативного сопровождения издательской деятельности вуза. Сотрудники подразделения принимают материалы от авторов, проверяют их соответствие установленным требованиям оформления, контролируют наличие обязательных структурных элементов, подготавливают замечания, формируют титульные листы, а также ведут учёт поступивших и утверждённых работ.

В условиях большого количества авторов и поступающих документов особую значимость приобретают скорость обработки материалов, точность проверки и удобство взаимодействия между преподавателями и сотрудниками отдела. Эффективность деятельности РИО напрямую влияет на своевременное обеспечение образовательного процесса актуальными учебными материалами.

1.2 Проблемы традиционного подхода к подготовке к изданию методических материалов

Традиционный подход к работе редакционно-издательского отдела, основанный на ручной обработке документов, сопровождается рядом существенных недостатков, снижающих эффективность работы подразделения:

1. Высокая трудоёмкость проверки оформления. Нормоконтроль почти всегда проводится вручную. Сотрудникам РИО приходится глазами выискивать ошибки в шрифтах, отступах, верстке таблиц и оформлении списка литературы. Когда поток работ большой, на эту монотонную проверку уходят десятки часов.

2. Высокая вероятность человеческих ошибок. Внимание проверяющего неизбежно притупляется из-за однотипных задач. В объемных текстах крайне легко пропустить сбившийся заголовок или неверный интервал. Из-за этого документ часто приходится возвращать на доработку в самый последний момент, перед выпуском.

3. Длительное взаимодействие с авторами. Найдя ошибки, редактор должен либо расписать их в отдельном письме, либо вручную выделять каждое нарушение в режиме рецензирования Word. На такие переписки и согласования правок тратится неоправданно много времени.

4. Отсутствие централизованного хранилища работ. Файлы оседают на личных компьютерах сотрудников или разрозненных сетевых дисках. Из-за этого бывает тяжело быстро поднять архивную копию, найти старую методичку или собрать данные по отделу.

5. Сложность формирования отчётности. Чтобы посчитать количество проверенных работ или оценить сроки обработки, данные приходится сводить из разных источников вручную. Это создает лишнюю нагрузку на персонал и задерживает получение метрик руководством.

6. Отсутствие унифицированных шаблонов. Преподаватели и авторы часто верстают титульные листы с нуля, не опираясь на актуальные требова-

ния. Как итог — постоянные отклонения от стандарта и бесконечные циклы исправлений одного и того же листа.

Таким образом, ручной подход к организации работы РИО не обеспечивает необходимой скорости, точности и удобства обработки документов в современных условиях.

1.3 Обоснование необходимости автоматизации

Решение перечисленных проблем возможно путём внедрения специализированной информационно-аналитической системы для автоматизации деятельности редакционно-издательского отдела вуза.

Разрабатываемая система должна обеспечить:

- загрузку пользователями учебно-методических и научных материалов через веб-интерфейс;
- автоматическую проверку документов на соответствие требованиям оформления;
- формирование подробного отчёта с перечнем найденных ошибок;
- автоматическое создание файла формата DOCX с выделенными нарушениями;
- автоматическое сохранение корректно оформленных работ в централизованном хранилище;
- ведение статистики поступивших, проверенных и утверждённых материалов;
- генерацию титульных листов по утверждённым шаблонам;
- разграничение прав доступа между авторами, редакторами и администраторами системы;
- удобный интерфейс взаимодействия пользователей с системой.

Автоматизация позволит существенно сократить время проверки материалов, снизить количество ошибок, ускорить процесс согласования документов и обеспечить прозрачный контроль деятельности подразделения.

1.4 Анализ существующих систем автоматизации документооборота и проверки документов

На современном рынке представлены различные программные решения, связанные с управлением документами, совместной работой и проверкой текстовых материалов. Однако большинство из них не ориентировано на специфику работы редакционно-издательского отдела вуза.

1.4.1 Microsoft Word и встроенные средства рецензирования

Текстовый редактор Microsoft Word широко применяется для подготовки и проверки документов. Он предоставляет следующие возможности:

- создание и редактирование документов;
- проверка орфографии и грамматики;
- режим рецензирования и внесения исправлений;
- использование шаблонов оформления.

Недостатком данного подхода является отсутствие автоматизированной проверки документа на соответствие локальным требованиям конкретного вуза. Проверка структуры, параметров оформления и соответствия внутренним стандартам выполняется вручную.

1.4.2 Google Docs

Google Docs является облачным сервисом для совместной работы с документами. Его возможности включают:

- онлайн-редактирование файлов;
- совместную работу нескольких пользователей;
- систему комментариев и предложений;
- хранение документов в облаке.

Несмотря на удобство совместного редактирования, сервис не предоставляет специализированных механизмов автоматической проверки учебно-методических материалов по требованиям университета, а также не содержит функций аналитической отчётности для РИО.

1.4.3 Системы электронного документооборота

Корпоративные системы электронного документооборота предназначены для регистрации, маршрутизации и хранения документов. Они позволяют:

- вести архив документов;
- организовывать согласование файлов;
- разграничивать права доступа;
- контролировать исполнение задач.

Однако подобные системы являются универсальными решениями и не включают инструменты интеллектуального анализа оформления документов DOCX, автоматического подчёркивания ошибок и генерации титульных листов.

1.4.4 Вывод по анализу аналогов

Проведённый анализ существующих решений показал, что универсальные офисные продукты и системы электронного документооборота способны решать лишь отдельные задачи редакционно-издательского отдела.

Одни системы обеспечивают редактирование документов, другие — хранение файлов и организацию согласования, однако ни одно из рассмотренных решений не сочетает в себе функции автоматической проверки оформления методических материалов, генерации отчётов об ошибках, формирования исправленного DOCX-файла и накопления статистики работы подразделения.

Следовательно, наиболее целесообразным является создание собственной специализированной информационно-аналитической системы, учитывающей особенности документооборота и нормативные требования конкретного высшего учебного заведения.

1.5 Постановка задач на разработку

На основании проведённого анализа были сформулированы следующие задачи, которые должна решать разрабатываемая система:

1. Обеспечить регистрацию и авторизацию пользователей с разграничением прав доступа.
2. Реализовать загрузку методических, учебных и научных материалов в формате DOCX.
3. Выполнить автоматическую проверку документов на соответствие установленным требованиям оформления.
4. Формировать отчёт с перечнем найденных нарушений.
5. Создавать копию документа DOCX с визуальным выделением ошибок.
6. Автоматически сохранять корректно оформленные работы в централизованной базе данных.
7. Реализовать хранение и поиск ранее загруженных материалов.
8. Предоставить средства формирования статистической и аналитической отчётности.
9. Реализовать модуль автоматического создания титульных листов по шаблону вуза.
10. Создать удобный веб-интерфейс для пользователей и административную панель для управления системой.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра "Информационно-аналитическая система для РИО вуза. Разработка сервиса "Методические указания"

2.2 Цель и назначение разработки

Основной целью работы является разработка и внедрение информационно-аналитической системы для автоматизации процессов проверки, хранения и сопровождения учебно-методических и научных материалов, поступающих в редакционно-издательский отдел вуза.

Информационно-аналитическая система предназначена для преподавателей, являющихся авторами учебно-методических разработок и сотрудников редакционно-издательского вуза.

2.3 Требования к функциональности

Информационная система должна обеспечивать выполнение следующих функций:

1. Управление пользователями. Система должна обеспечивать регистрацию и авторизацию пользователей по логину и паролю. Должно быть реализовано разграничение прав доступа: администратор, редактор, пользователь. Администратор должен иметь возможность управления учётными записями пользователей.

2. Загрузка документов. Пользователь должен иметь возможность загружать методические материалы, учебные пособия, монографии и иные документы в формате DOCX через веб-интерфейс системы.

3. Автоматическая проверка оформления. После загрузки документа система должна автоматически анализировать файл на соответствие установленным требованиям оформления. Проверка должна включать контроль

структуры документа, параметров шрифта, интервалов, отступов, заголовков, таблиц, изображений, списков литературы и иных элементов.

4. Формирование отчёта об ошибках. По завершении анализа система должна формировать отчёт, содержащий перечень обнаруженных нарушений, описание ошибок и рекомендации по их устранению.

5. Создание документа с выделением ошибок. Система должна создавать копию загруженного документа в формате DOCX, в которой найденные ошибки визуально выделены средствами форматирования.

6. Хранение корректных документов. Если загруженный документ соответствует всем требованиям, он должен автоматически сохраняться в централизованной базе данных или файловом архиве системы.

7. Поиск и просмотр документов. Система должна обеспечивать просмотр списка ранее загруженных материалов, поиск по названию, автору, дате загрузки, статусу проверки и другим параметрам.

8. Генерация титульных листов. Пользователь должен иметь возможность заполнить форму с исходными данными, после чего система должна автоматически сформировать титульный лист по утверждённому шаблону вуза.

9. Статистика и аналитика. Система должна формировать сводные данные по количеству загруженных документов, числу успешных проверок, количеству найденных ошибок, активности пользователей и временным показателям обработки материалов.

10. Логирование действий. Система должна сохранять сведения о действиях пользователей: загрузка файлов, проверка документов, вход в систему, генерация титульных листов и иные операции.

2.4 Требования к интерфейсу

Интерфейс информационной системы представляет собой веб-приложение с единым современным стилем оформления, обеспечивающее удобную и интуитивно понятную навигацию. Пользовательский интерфейс

должен быть адаптирован для работы в распространённых браузерах и обеспечивать быстрый доступ к основным функциям системы.

Основные элементы интерфейса включают:

- главную страницу системы;
- форму авторизации и регистрации пользователей;
- личный кабинет пользователя;
- страницу загрузки документов на проверку;
- страницу просмотра результатов проверки;
- страницу архива сохранённых документов;
- страницу генерации титульных листов;
- административную панель управления пользователями;
- модуль просмотра статистики и аналитических данных.

На рисунке 2.1 представлена страница авторизации системы.

Рисунок 2.1 – Страница авторизации

Как показано на рисунке, форма авторизации содержит поля ввода логина и пароля, кнопку входа в систему, а также элементы перехода к регистрации нового пользователя. После успешной авторизации пользователь получает доступ к функциональным возможностям системы в соответствии со своей ролью.

На рисунке 2.2 представлен внешний вид личного кабинета пользователя.

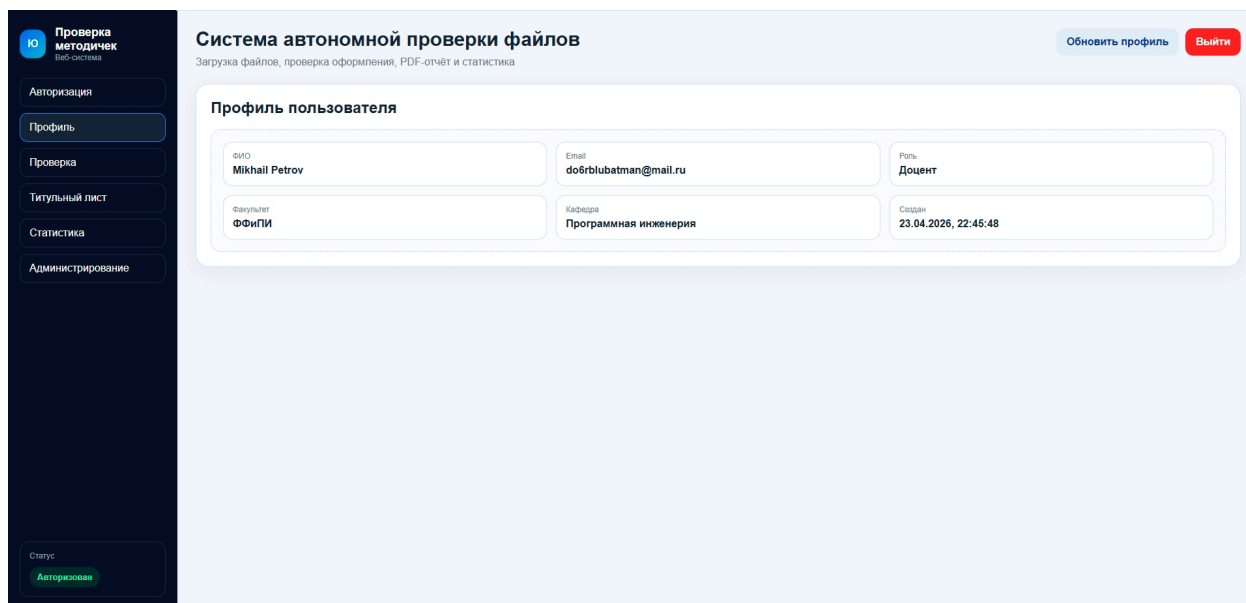


Рисунок 2.2 – Личный кабинет пользователя

Личный кабинет содержит навигационное меню, сведения о текущем пользователе, основные разделы системы, а также быстрый доступ к загрузке документов, архиву файлов и генерации титульных листов.

На рисунке 2.3 представлена страница проверки документов.

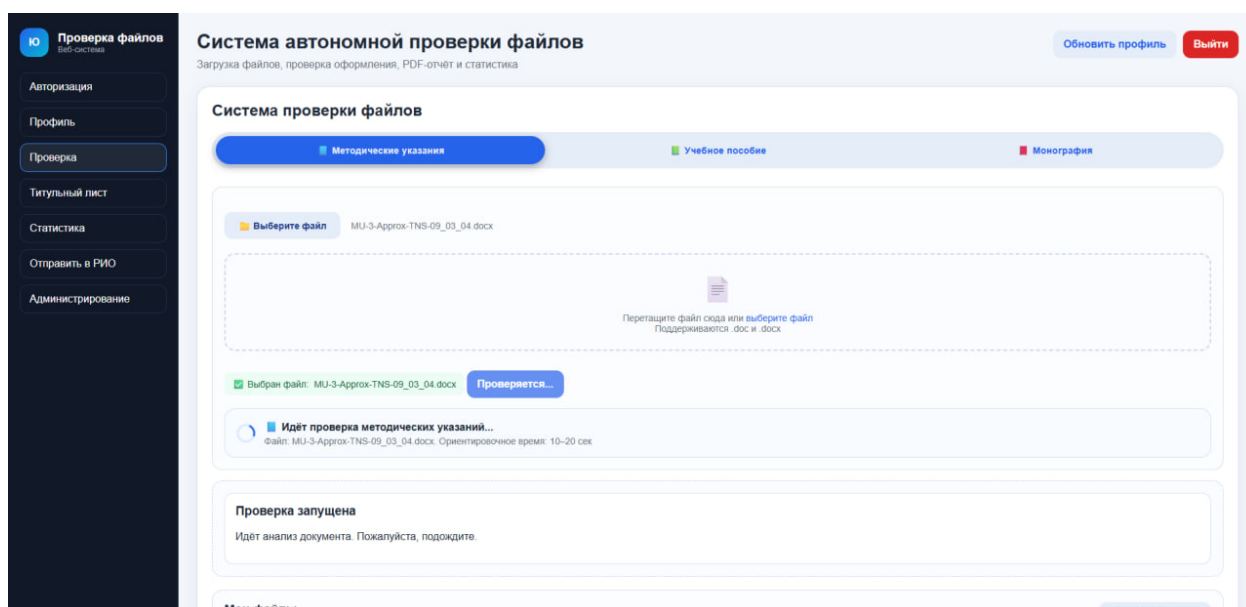


Рисунок 2.3 – Страница проверки документов

Данная страница содержит форму выбора файла формата DOCX, кнопку запуска автоматической проверки, область отображения текущего статуса обработки и результаты анализа после завершения проверки.

На рисунке 2.4 представлена страница генерации титульных листов.

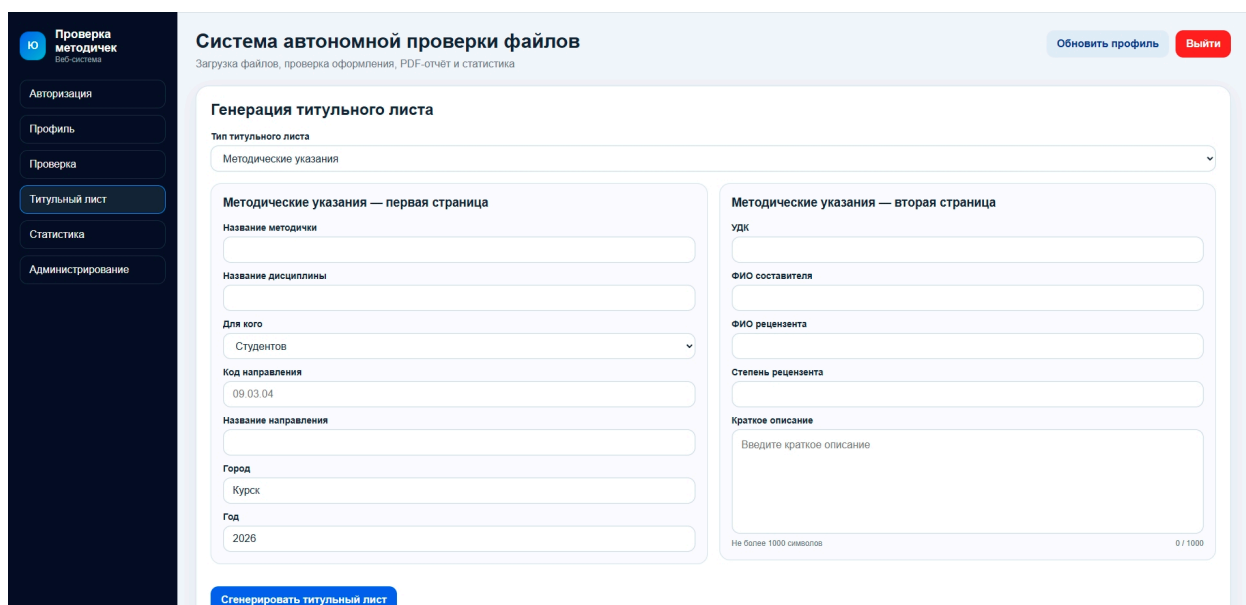


Рисунок 2.4 – Страница генерации титульных листов

Как показано на рисунке, страница содержит форму ввода реквизитов работы: наименование дисциплины, тема работы, сведения об авторе, руко-

водителе и кафедре. После заполнения формы пользователь может сформировать готовый документ в формате DOCX.

На рисунке 2.5 представлена страница статистики и аналитики.

Рисунок 2.5 – Страница статистики

Страница статистики содержит сведения о количестве зарегистрированных пользователей, числе загруженных документов, результатах проверок, количестве обнаруженных ошибок, а также иные аналитические показатели деятельности системы.

Интерфейс системы должен обеспечивать логичное расположение элементов управления, единообразие оформления страниц, наглядность представления результатов работы и минимальное количество действий пользователя при выполнении основных операций.

2.5 Требования к обработке ошибок

Система должна обеспечивать корректную обработку исключительных ситуаций.

Основные требования:

- при возникновении ошибки пользователю должно отображаться понятное сообщение;

- внутренние ошибки системы не должны раскрывать служебную информацию;
- ошибки должны фиксироваться в журнале событий;
- должна быть реализована обработка:
 - некорректных входных данных;
 - ошибок загрузки файлов;
 - ошибок обработки документов;
 - ошибок взаимодействия с базой данных.

2.6 Требования к удобству использования

Система должна обеспечивать удобство и простоту взаимодействия пользователей.

Основные требования:

- интерфейс должен быть интуитивно понятным;
- количество действий для выполнения основной операции (загрузка и проверка документа) должно быть минимальным;
- результаты проверки должны отображаться в структурированном виде;
- должна быть реализована визуальная обратная связь (индикаторы загрузки, статусы операций).

2.7 Моделирование вариантов использования

Для определения функциональных возможностей разрабатываемой системы и взаимодействия пользователей с её компонентами была выполнена модель вариантов использования.

В системе предусмотрены следующие роли пользователей:

Администратор — обладает полными правами доступа, включая управление пользователями, просмотр статистики, настройку системы, удаление документов и контроль работы пользователей.

Редактор — осуществляет просмотр загруженных материалов, контроль результатов автоматической проверки, работу с архивом документов и формирование отчётности.

Пользователь — выполняет загрузку документов, получает результаты проверки, скачивает отчёты, использует модуль генерации титульных листов и просматривает историю своих материалов.

Основные прецеденты (варианты использования) системы:

1. Вход в систему.
2. Регистрация пользователя.
3. Загрузка документа на проверку.
4. Автоматическая проверка оформления.
5. Просмотр отчёта об ошибках.
6. Скачивание исправленного документа.
7. Сохранение корректного документа в архив.
8. Поиск и просмотр ранее загруженных работ.
9. Генерация титульного листа.
10. Просмотр статистики работы системы.
11. Управление пользователями.
12. Выход из системы.

Для повышения наглядности диаграмма прецедентов была разделена по ролям пользователей. Отдельно представлены варианты использования для пользователя, администратора и редактора.

На рисунке 2.6 представлена диаграмма прецедентов пользователя.

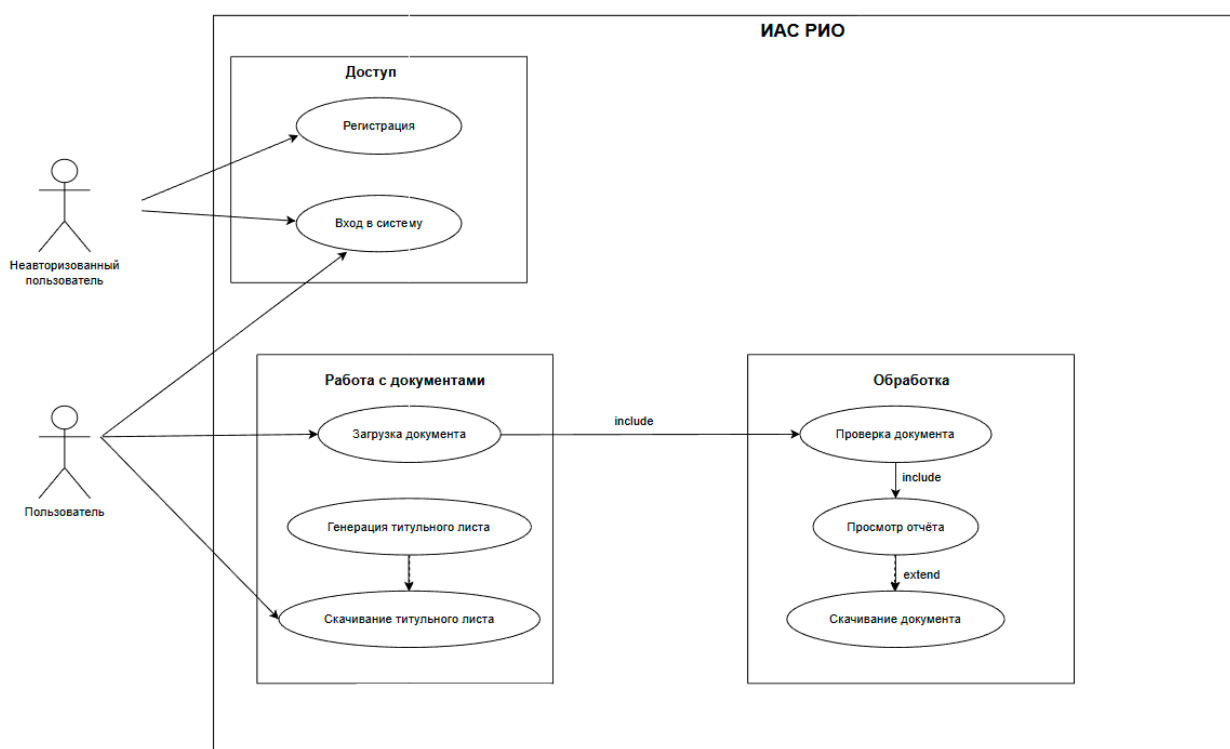


Рисунок 2.6 – Диаграмма прецедентов пользователя

На рисунке 2.7 представлена диаграмма прецедентов администратора.



Рисунок 2.7 – Диаграмма прецедентов администратора

На рисунке 2.8 представлена диаграмма прецедентов редактора.

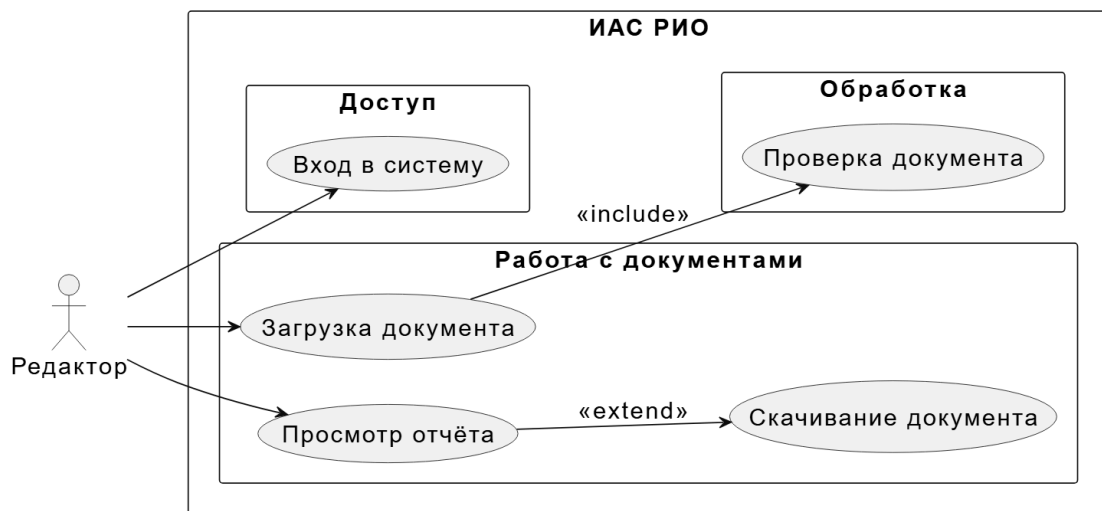


Рисунок 2.8 – Диаграмма прецедентов редактора

2.7.1 Вариант использования «Вход в систему»

Заинтересованные лица и их требования: пользователь желает получить доступ к функциям системы путём ввода логина и пароля.

Предусловие: пользователь не авторизован; открыта страница входа.

Постусловие: пользователь успешно авторизован и перенаправлен в личный кабинет либо на главную страницу системы.

Основной успешный сценарий:

1. Пользователь открывает страницу входа.
2. Вводит логин и пароль.
3. Нажимает кнопку «Войти».
4. Система выполняет поиск пользователя в базе данных.
5. Выполняется проверка пароля.
6. Создаётся пользовательская сессия.
7. Пользователь получает доступ к функциям системы.

Альтернативный сценарий (неверные данные):

1. Пользователь вводит неверный логин или пароль.
2. Система отображает сообщение об ошибке авторизации.
3. Пользователь остаётся на странице входа.

Альтернативный сценарий (пустые поля):

1. Пользователь не заполняет одно из обязательных полей.
2. Система выводит сообщение о необходимости заполнения формы.

2.7.2 Вариант использования «Выход из системы»

Заинтересованные лица и их требования: авторизованный пользователь желает завершить работу с системой.

Предусловие: пользователь авторизован.

Постусловие: пользовательская сессия завершена.

Основной успешный сценарий:

1. Пользователь нажимает кнопку «Выход».
2. Система удаляет активную сессию.
3. Пользователь перенаправляется на страницу входа.

2.7.3 Вариант использования «Загрузка документа на проверку»

Заинтересованные лица и их требования: пользователь желает отправить документ в формате DOCX для автоматической проверки.

Предусловие: пользователь авторизован.

Постусловие: файл загружен в систему и передан модулю проверки.

Основной успешный сценарий:

1. Пользователь открывает страницу загрузки.
2. Выбирает файл формата DOCX.
3. Нажимает кнопку «Загрузить».
4. Система сохраняет файл во временное хранилище.
5. Запускается процедура проверки документа.

Альтернативный сценарий (неподдерживаемый формат):

1. Пользователь выбирает файл иного формата.
2. Система выводит сообщение об ошибке.
3. Загрузка не выполняется.

Альтернативный сценарий (файл не выбран):

1. Пользователь нажимает кнопку загрузки без выбора файла.

2. Система предлагает выбрать документ.

2.7.4 Вариант использования «Автоматическая проверка документа»

Заинтересованные лица и их требования: пользователь хочет увидеть, где именно в его тексте нарушены стандарты оформления.

Предусловие: файл загружен на сервер и доступен для чтения.

Постусловие: алгоритм сгенерировал итоговую сводку по проверке.

Основной успешный сценарий:

1. Парсер системы разбирает внутреннюю структуру файла.
2. Выполняется проверка параметров оформления.
3. Модуль фиксирует все отклонения от заданных требований.
4. Платформа собирает найденные дефекты в единый список.

Альтернативный сценарий (ошибка обработки файла):

1. Документ повреждён либо содержит некорректную структуру.
2. Сервер принудительно останавливает процесс проверки.
3. В интерфейсе всплывает уведомление о невозможности прочитать документ.

2.7.5 Вариант использования «Получение отчёта об ошибках»

Заинтересованные лица и их требования: пользователь желает ознакомиться с найденными нарушениями.

Предусловие: проверка завершена.

Постусловие: пользователю отображён отчёт.

Основной успешный сценарий:

1. После завершения проверки система формирует отчёт.
2. Пользователь открывает страницу результата.
3. Отображается список ошибок с пояснениями.

Альтернативный сценарий (ошибок не обнаружено):

1. Проверка завершена без замечаний.
2. Система отображает сообщение о корректности документа.

2.7.6 Вариант использования «Скачивание документа с выделенными ошибками»

Заинтересованные лица и их требования: пользователь желает скачать исправляемый файл с визуально отмеченными ошибками.

Предусловие: документ содержит нарушения.

Постусловие: пользователю предоставлен файл DOCX для скачивания.

Основной успешный сценарий:

1. Пользователь нажимает кнопку скачивания.
2. Система формирует копию документа.
3. Ошибки подчёркиваются или выделяются.
4. Пользователь скачивает файл.

Альтернативный сценарий (файл не сформирован):

1. Во время генерации файла возникает ошибка.
2. Система выводит сообщение о невозможности подготовки документа.

2.7.7 Вариант использования «Автоматическое сохранение корректного документа»

Заинтересованные лица и их требования: пользователь желает сохранить корректно оформленную работу в архиве системы.

Предусловие: ошибок в документе не обнаружено.

Постусловие: файл помещён в централизованное хранилище.

Основной успешный сценарий:

1. Проверка завершается без ошибок.
2. Система сохраняет документ в архив.
3. Информация о работе заносится в базу данных.

Альтернативный сценарий (ошибка сохранения):

1. Возникает ошибка доступа к хранилищу либо базе данных.
2. Система фиксирует ошибку в журнале событий.

3. Пользователю выводится уведомление.

2.7.8 Вариант использования «Просмотр архива документов»

Заинтересованные лица и их требования: пользователю необходим доступ к истории ранее проверенных файлов.

Предусловие: сессия пользователя активна (пройдена авторизация).

Постусловие: на странице отрисована таблица с историей загрузок.

Основной успешный сценарий:

1. Пользователь переходит во вкладку архивных записей.
2. Сервер извлекает из базы данных перечень прошлых проверок.
3. Пользователь находит интересующий материал и сохраняет его на локальный диск.

Альтернативный сценарий (архив пуст):

1. В базе данных нет ни одной записи, привязанной к текущему профилю.
2. Интерфейс выводит экранную заглушку с текстом «Архив пуст».

2.7.9 Вариант использования «Формирование титульного листа»

Заинтересованные лица и их требования: пользователь желает автоматически создать титульный лист по шаблону вуза.

Предусловие: пользователь авторизован.

Постусловие: сформирован титульный лист.

Основной успешный сценарий:

1. Пользователь открывает форму генерации титульного листа.
2. Заполняет необходимые поля.
3. Нажимает кнопку формирования.
4. Система создаёт документ DOCX.

Альтернативный сценарий (не заполнены обязательные поля):

1. Пользователь оставляет обязательные поля пустыми.
2. Система сообщает о необходимости заполнения данных.

2.7.10 Вариант использования «Просмотр статистики»

Заинтересованные лица и их требования: администратор желает получить статистические сведения о работе системы.

Предусловие: администратор авторизован.

Постусловие: отображён аналитический отчёт.

Основной успешный сценарий:

1. Администратор открывает раздел статистики.
2. Система подсчитывает показатели.
3. Отображаются данные по количеству проверок, ошибок, сохранённых документов и пользователям.

Альтернативный сценарий (недостаточно прав):

1. Обычный пользователь пытается открыть раздел статистики.
2. Система запрещает доступ.

2.7.11 Вариант использования «Управление пользователями»

Заинтересованные лица и их требования: администратор желает управлять учётными записями пользователей.

Предусловие: администратор авторизован.

Постусловие: учётные записи обновлены.

Основной успешный сценарий:

1. Администратор открывает раздел пользователей.
2. Добавляет, редактирует или удаляет записи.
3. Система сохраняет изменения.

Альтернативный сценарий (дублирование логина):

1. При создании пользователя введён уже существующий логин.
2. Система выводит сообщение об ошибке.

2.7.12 Вариант использования «Настройка параметров системы»

Заинтересованные лица и их требования: администратор желает изменить правила проверки документов и параметры работы системы.

Предусловие: учетная запись обладает правами администратора.

Постусловие: новые параметры конфигурации сохранены в систему.

Основной успешный сценарий:

1. Администратор переходит в панель управления системой.
2. Изменяет параметры.
3. Подтверждает сохранение через пользовательский интерфейс.
4. Система принимает новые значения.

Альтернативный сценарий (некорректные параметры):

1. В формы ввода переданы значения, нарушающие бизнес-логику.
2. Система отменяет сохранение и выводит сообщение об ошибке

2.8 Требования к надёжности

Спроектированная информационно-аналитическая система обязана выдерживать пиковые нагрузки без программных сбоев, а также полностью исключать риск потери загруженных файлов или сгенерированных метрик.

Целостность данных. Любые изменения состояния платформы — будь то загрузка нового документа, логирование результатов проверки, создание учетной записи или экспорт статистики — должны выполняться строго в рамках атомарных транзакций. При возникновении ошибки операция должна быть отменена без повреждения существующих данных.

Устойчивость к сбоям. При аварийном завершении работы сервера, отключении электропитания или программном сбое система должна сохранять ранее записанные данные. После повторного запуска система должна продолжить работу в штатном режиме.

Контроль корректности входных данных. На уровне приложения должна выполняться проверка корректности вводимых пользователями данных: обязательность заполнения полей, допустимые форматы файлов, ограничения длины строковых значений, корректность дат и иных параметров.

Логирование ошибок. Критические ошибки, исключения и события работы системы должны фиксироваться в журнале событий. Записи журнала должны содержать дату, время, уровень важности и описание ошибки.

Резервное копирование. База данных и архив документов должны допускать резервное копирование штатными средствами используемой СУБД и файловой системы. Восстановление данных должно выполняться из резервной копии.

2.9 Требования к информационной безопасности

Система должна обеспечивать защиту данных от несанкционированного доступа и разграничение прав пользователей.

Авторизация пользователей. Доступ к функциям системы должен предоставляться только после успешного ввода логина и пароля.

Хранение паролей. Пароли пользователей не должны храниться в открытом виде. Для хранения необходимо использовать криптографическое хеширование.

Разграничение прав доступа. В системе должны поддерживаться роли пользователей:

- пользователь — загрузка документов, просмотр результатов, генерация титульных листов;
- администратор — полный доступ, включая управление пользователями и просмотр статистики.

Проверка доступа на серверной стороне. Права пользователя должны проверяться сервером при выполнении каждого защищённого действия.

Защита от SQL-инъекций. Доступ к базе данных должен осуществляться с использованием ORM либо параметризованных запросов.

Защита файлового хранилища. Загруженные документы и архивные материалы должны храниться в каталогах, недоступных для прямого неавторизованного обращения.

2.10 Требования к программной и технической совместимости

Система должна функционировать в типовой вычислительной среде и не предъявлять завышенных требований к аппаратным ресурсам.

Операционная система сервера:

- Windows 10/11.

Аппаратные требования:

- процессор с тактовой частотой от 2 ГГц;
- оперативная память не менее 4 ГБ;
- свободное дисковое пространство от 1 ГБ без учёта пользовательских файлов.

Клиентская часть. Для работы пользователей необходимы текстовый редактор Word и современный веб-браузер:

- Яндекс Браузер;
- Google Chrome;
- Mozilla Firefox;
- Microsoft Edge.

2.11 Требования к оформлению документации

Разработка программной документации и программного изделия должна выполняться в соответствии с требованиями действующих нормативных документов ГОСТ 19.102–77 и ГОСТ 34.601–90. Единая система программной документации.

3 Технический проект

3.1 Архитектура программной системы

Разрабатываемая информационно-аналитическая система построена по архитектуре «клиент-сервер» с использованием REST-подхода.

Серверная часть реализована на языке Python с использованием фреймворка FastAPI и обеспечивает обработку запросов, выполнение бизнес-логики, а также взаимодействие с базой данных PostgreSQL и файловым хранилищем документов.

Клиентская часть представляет собой веб-приложение, доступ к которому осуществляется через браузер. Взаимодействие между клиентом и сервером выполняется по протоколу HTTP с использованием формата JSON для обмена данными.

Архитектура системы включает следующие основные компоненты:

- клиентское веб-приложение (интерфейс пользователя);
- серверное приложение (FastAPI);
- система управления базами данных PostgreSQL;
- файловое хранилище для загружаемых и сгенерированных документов;
- модуль анализа и проверки документов формата DOCX.

Логика работы системы организована следующим образом. Пользователь через веб-интерфейс выполняет действия (авторизация, загрузка документа, запуск проверки и др.), которые инициируют HTTP-запросы к серверу. Сервер обрабатывает запрос, при необходимости обращается к базе данных и файловой системе, выполняет проверку документа и возвращает результат клиенту.

Модуль проверки документов осуществляет анализ структуры и оформления файлов формата DOCX, выявляет несоответствия установленным требованиям и формирует отчёт об ошибках. При необходимости система генерирует исправленный файл с визуальным выделением нарушений.

Все загружаемые документы сохраняются в файловой системе, а информация о них (путь к файлу, результаты проверки, пользователь и др.) — в базе данных PostgreSQL. Это обеспечивает разделение данных и повышает масштабируемость системы.

Для обеспечения безопасности и управления доступом используется механизм аутентификации пользователей. После успешного входа пользователь получает токен доступа, который используется при последующих запросах к API.

Общая схема взаимодействия компонентов системы представлена на рисунке 3.1.

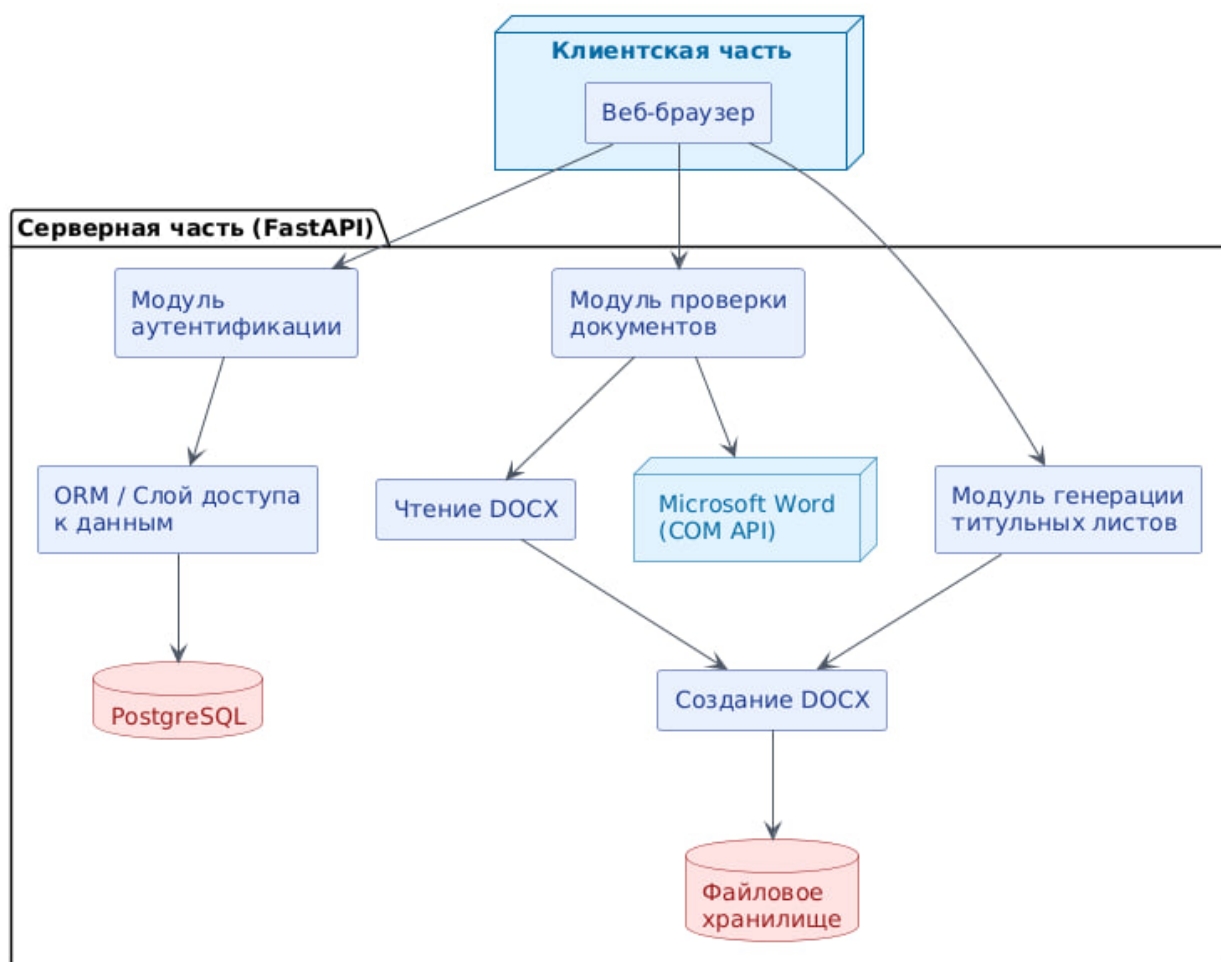


Рисунок 3.1 – Общая схема архитектуры системы

3.2 Модуль обработки учебно-методических документов

Модуль обработки учебно-методических документов является центральным компонентом системы, отвечающим за приём, анализ и проверку файлов формата DOCX. Он одинаково работает для методических указаний, учебных пособий и монографий, поскольку алгоритм проверки оформления и механизмы работы с документами универсальны. Различия между типами документов проявляются только на этапе генерации титульных листов, где используются разные наборы полей и шаблоны оформления.

3.2.1 Общая архитектура модуля

Модуль построен по многоуровневой архитектуре и включает следующие компоненты:

- **слой маршрутизации** — приём HTTP-запросов, валидация входных данных, маршрутизация к соответствующим обработчикам;
- **сервисный слой** — реализация бизнес-логики: проверка документов, генерация титульных листов, расчёт статистики;
- **слой доступа к данным** — объектно-реляционное отображение для взаимодействия с PostgreSQL;
- **слой утилит** — вспомогательные функции для работы с файлами, хеширования, логирования.

Такое разделение обеспечивает слабую связанность компонентов и упрощает тестирование и дальнейшее расширение.

3.2.2 Интеграция с Microsoft Word через COM-интерфейс

Ключевой особенностью модуля является использование COM-интерфейса Microsoft Word через библиотеку pywin32. При запуске проверки создаётся экземпляр приложения Word в фоновом режиме с отключёнными всплывающими окнами и диалогами. Это позволяет выполнять анализ и модификацию документов без видимого запуска графического интерфейса Word.

Основные операции, выполняемые через СОМ-интерфейс:

- открытие документа (`Documents.Open`) с возможностью только чтения или с правом записи;
- получение общей информации о документе: количество страниц, разделов, абзацев;
- обход всех абзацев документа и извлечение параметров форматирования;
- доступ к структуре страниц (поля, колонтитулы, нумерация);
- модификация документа: изменение цвета шрифта для выделения ошибок, добавление комментариев;
- сохранение изменённой копии и закрытие документа.

3.2.3 Алгоритм автоматической проверки оформления

Процесс проверки запускается после получения файла и его временно-го сохранения. Алгоритм реализован в классе `WordMethodicalChecker` и включает следующие этапы.

Этап 1. Предварительная обработка.

Файл сохраняется во временную директорию, создаётся уникальный идентификатор задачи для отслеживания прогресса. Если требуется сохранение исправленной версии (параметр `mark_document = True`), создаётся копия документа с суффиксом `_checked`, с которой будет работать модуль. Исходный файл остаётся неизменным.

Этап 2. Определение объёма работы.

Через СОМ-интерфейс открывается документ, вычисляется общее количество страниц с помощью метода `ComputeStatistics(WD_STATISTIC_PAGES)`. Эта информация используется для последующей группировки ошибок по страницам.

Этап 3. Фильтрация служебных элементов.

При обходе абзацев пропускаются:

- пустые абзацы — определяются по длине текста (менее 3 символов);

– абзацы, находящиеся внутри таблиц — метод `p.Range.Information(WD_WITHIN_TABLE)` возвращает `True` для таких элементов;

– заголовки — определяются по имени стиля: если в названии стиля присутствует подстрока "heading" или "заголов", абзац считается заголовком и пропускается.

Отфильтрованные элементы не участвуют в сравнении параметров оформления, так как к ним предъявляются иные требования или они не являются частью основного текста.

Этап 4. Извлечение параметров форматирования.

Для каждого значимого абзаца извлекаются следующие параметры:

– **шрифт** — `p.Range.Font.Name`, эталонное значение "Times New Roman";

– **размер шрифта** — `p.Range.Font.Size`, эталонное значение 16 pt;

– **отступ первой строки** — `p.Range.ParagraphFormat.FirstLineIndent`, сначала система анализирует форматирование текущего абзаца. В случае, когда обнаруженное значение близко к нулю, выполняется обращение к стилю абзаца для получения корректного отступа.

– **межстрочный интервал** — `p.Range.ParagraphFormat.LineSpacingRule`, эталонное значение `WD_LINE_SPACE_SINGLE` (одинарный интервал);

– **выравнивание текста** — анализируется при проверке номеров страниц.

Особое внимание уделяется случаю, когда в одном абзаце встречается неоднородное форматирование (например, разные размеры шрифта в одной строке). В таких ситуациях выполняется дополнительный обход всех слов внутри абзаца через коллекцию `p.Range.Words`. Для каждого слова извлекается его размер шрифта `w.Font.Size`. Если размер отличается от эталонного значения 16 pt более чем на 0,2 pt, и при этом слово не является знаком препинания (точка, запятая, скобка и т.п.), фиксируется ошибка «неверный размер шрифта» с указанием конкретного слова и номера страницы. Значения размера, равные 9999999,0, интерпретируются как ошибочные (такое значение возникает при попытке прочитать размер шрифта у объекта, не имеющего единого форматирования) и также приводят к фиксации нарушения. Такой

подход позволяет выявить ситуации, когда большая часть абзаца отформатирована правильно, но отдельные слова имеют неверный кегль, что часто встречается при копировании текста из внешних источников.

Этап 5. Сравнение с эталонами и фиксация ошибок.

Полученные параметры преобразуются в сантиметры (для отступов) и сравниваются с эталонными значениями:

- для отступа используется допуск 0,05 см: $\text{abs}(\text{indent_cm} - 1.25) > 0.05$;
- для размера шрифта используется допуск 0,2 pt: $\text{abs}(\text{size} - 16) > 0.2$;
- для шрифта и интервала выполняется точное сравнение (без допуска).

При обнаружении отклонения фиксируется номер страницы, тип нарушения и поясняющее сообщение. Номер страницы получается методом `p.Range.Information(WD_ACTIVE_END_PAGE_NUMBER)`. Каждая ошибка сохраняется в виде объекта `CheckIssue`, содержащего:

- `rule` — раздел проверки (например, "3" — для параметров текста, "4" — для нумерации);
- `severity` — степень важности ("ERROR" или "WARNING");
- `location` — местоположение в документе (страница или диапазон страниц);
- `message` — текст сообщения с описанием нарушения и рекомендацией по исправлению;
- `priority` — приоритет (используется для сортировки).

Этап 6. Проверка полей страницы.

После обработки всех абзацев выполняется проверка параметров страниц. Для каждого раздела документа (коллекция `doc.Sections`) извлекаются и проверяются настройки `PageSetup`:

- верхнее поле (`TopMargin`) — эталон 3,0 см;
- нижнее поле (`BottomMargin`) — эталон 2,25 см;
- левое поле (`LeftMargin`) — эталон 2,2 см;
- правое поле (`RightMargin`) — эталон 2,3 см.

Все поля проверяются в сантиметрах с допуском 0,05 см. При нарушении фиксируется номер раздела, и ошибка добавляется в отчёт.

Этап 7. Проверка нумерации страниц.

Проверка выполняется по всем разделам документа. Для каждого раздела анализируются верхний (Headers(WD_HEADER_FOOTER_PRIMARY)) и нижний (Footers(WD_HEADER_FOOTER_PRIMARY)) колонтитулы:

1. Если в верхнем колонтитуле обнаружена поле с типом WD_FIELD_PAGE — нумерация присутствует в верхней части страницы, дополнительно проверяется выравнивание номера (должно быть по центру, WD_ALIGN_CENTER).

2. Если номер найден только в нижнем колонтитуле — фиксируется ошибка о неверном расположении номера страницы.

3. Если номер не найден ни в верхнем, ни в нижнем колонтитуле — фиксируется ошибка об отсутствии нумерации страниц.

Этап 8. Группировка ошибок.

Для повышения удобочитаемости отчёта последовательные страницы с одинаковыми типами ошибок объединяются в диапазоны. Например, если ошибки шрифта обнаружены на страницах 5, 6, 7, 8, отчёт покажет «Стр. 5–8» вместо четырёх отдельных записей. Группировка выполняется методом `_group_pages`, который сортирует номера страниц, находит в них непрерывные последовательности и формирует соответствующие записи. Для каждого диапазона создаётся одна запись в отчёте, а для всех страниц диапазона выполняется визуальное выделение ошибок.

Этап 9. Завершение проверки и освобождение ресурсов.

По окончании обхода всех абзацев и проверки полей и нумерации страниц выполняется сохранение изменений в документе (если `mark_document = True`) и закрытие документа через `doc.Close(False)`. После этого экземпляр Word завершается вызовом `self.word.Quit(False)`. Все временные файлы, созданные в процессе проверки (временная копия документа, файлы для отчёта), удаляются. Если на любом из этапов произошла ошибка, Word принудительно за-

вершается, а временные файлы удаляются в блоке `finally`, что гарантирует отсутствие «висящих» процессов и освобождение дискового пространства.

3.2.4 Формирование исправленной копии документа

Если в документе обнаружены ошибки и установлен флаг `mark_document` (активируется по умолчанию), модуль создаёт копию, в которой проблемные фрагменты визуально выделены. Алгоритм выделения работает следующим образом:

1. Для каждой группы ошибок определяется диапазон страниц, на которых они обнаружены.
2. Для первой страницы диапазона вычисляется начальная позиция через `doc.GoTo(1, 1, page)`. Константа 1 обозначает переход к странице, второй параметр — тип объекта (страница).
3. Для последней страницы диапазона вычисляется конечная позиция через `doc.GoTo(1, 1, page + 1)`. Если страница последняя в документе, конечная позиция определяется как `doc.Content.End`.
4. Область выделения задаётся через `doc.Range(start, end)`.
5. Цвет шрифта в выделенной области устанавливается в красный (`WD_COLOR_RED`) с помощью свойства `rng.Font.Color`.
6. К выделенной области добавляется комментарий с поясняющим сообщением через `doc.Comments.Add(rng, message)`.

Такой подход позволяет сохранить исходную структуру документа и одновременно наглядно показать автору все места, требующие исправления. Комментарии Word видны при наведении курсора и не нарушают вёрстку документа.

3.2.5 Генерация PDF-отчёта

Параллельно с модификацией документа формируется PDF-отчёт с использованием библиотеки `reportlab`. Процесс создания отчёта включает следующие шаги:

1. Регистрация шрифтов семейства Times New Roman (обычный, жирный, курсив, жирный курсив) для корректного отображения кириллицы. Библиотека reportlab не поддерживает кириллицу по умолчанию, поэтому требуется явная регистрация шрифтов с указанием пути к файлам .ttf.

2. Настройка параметров страницы: формат A4, поля 15 мм со всех сторон.

3. Формирование заголовочной части: название документа («Отчёт по проверке»), имя проверенного файла, дата и время проверки.

4. Определение итогового статуса:

- если есть ошибки уровня ERROR — статус «документ содержит критические ошибки» (отображается красным цветом);

- если есть только предупреждения (WARNING) — «документ содержит предупреждения» (оранжевый);

- если ошибок нет — «документ соответствует требованиям» (зелёный).

5. Построение таблицы ошибок с колонками: номер ошибки, раздел проверки, тип (ошибка/предупреждение), местоположение в документе, описание проблемы.

6. Стилизация таблицы: строки с ошибками выделяются красным фоном (BackgroundColor = #FEE2E2), строки с предупреждениями — жёлтым (BackgroundColor = #FEF3C7).

Отчёт сохраняется в директорию reports/ с именем, содержащим уникальный идентификатор, чтобы избежать конфликтов при одновременной работе нескольких пользователей.

3.2.6 Генерация титульных листов

Генерация титульных листов реализована через библиотеку python-docx без использования Microsoft Word. Это позволяет выполнять операцию быстро и без зависимости от установленного офисного пакета. В зависимости от типа документа используются разные наборы полей и шаблоны:

1. Для методических указаний: название, дисциплина, направление подготовки, составитель, рецензенты (с возможностью добавления нескольких), УДК, описание.

2. Для учебных пособий: автор, название, рецензенты, направления подготовки, значение А, ISBN, УДК, ББК, описание.

3. Для монографии: авторы (с возможностью добавления нескольких), название, город, год, УДК, ББК, ISBN, описание.

Алгоритм генерации единый для всех типов:

1. Создаётся новый пустой документ DOCX.
2. Устанавливаются поля страницы: верхнее 3 см, нижнее 2,25 см, левое 2,2 см, правое 2,3 см.
3. Задаётся стиль Normal: шрифт Times New Roman, размер 16 pt.
4. Последовательно добавляются абзацы с заданным выравниванием (центр, право, лево) и форматированием.
5. Для каждого абзаца через вспомогательную функцию `add_paragraph` устанавливаются параметры: текст, выравнивание, размер шрифта, жирное начертание, отступы до и после абзаца, межстрочный интервал.
6. После заполнения первой страницы добавляется разрыв раздела (`doc.add_section(WD_SECTION_START.NEW_PAGE)`), и формируется вторая страница.
7. Вторая страница содержит библиографическое описание, аннотацию и выходные данные — в зависимости от типа документа набор полей различается.
8. Готовый документ сохраняется во временную директорию, после чего отправляется пользователю и удаляется с сервера.

3.2.7 Управление сессиями и обновление токенов

Для поддержания авторизации пользователя без многократного ввода пароля используется двухуровневая система токенов: `access`-токен (срок действия 60 минут) и `refresh`-токен (срок действия 30 дней). `Access`-токен подписывается секретным ключом и содержит идентификатор пользователя и его

email. Refresh-токен генерируется как случайная строка, его хеш сохраняется в базе данных вместе с метаданными (IP-адрес, User-Agent).

При получении запроса к защищённому ресурсу сервер проверяет access-токен: если он истёк, клиент получает ошибку 401 Unauthorized и автоматически отправляет refresh-токен на эндпоинт /auth/refresh. Сервер проверяет, что refresh-токен не отозван (поле revoked), не истёк и принадлежит тому же пользователю, после чего выдаёт новый access-токен. Refresh-токен при этом остаётся без изменений, что позволяет обновлять access-токен многократно без повторного входа пользователя.

Если пользователь выходит из системы, то соответствующий refresh-токен в базе данных получает признак отзыва (revoked = True). Такой токен уже не может быть использован повторно. При подозрении на компрометацию учётной записи администратор вправе одной командой отозвать все токены пользователя.

3.2.8 Взаимодействие с базой данных через ORM

Для взаимодействия с PostgreSQL используется объектно-реляционное отображение SQLAlchemy. ORM позволяет работать с записями базы данных как с обычными объектами Python, скрывая детали формирования SQL-запросов. Каждая таблица описывается отдельным классом, наследующим от Base, с указанием типов полей, ограничений и связей.

Пример описания таблицы manual:

```
class Manual(Base):
    __tablename__ = "manual"
    id_manual = Column(Integer, primary_key=True, index=True)
    manual_name = Column(String(255), nullable=False)
    fio_user = Column(String(255), nullable=False)
    faculty_code = Column(String(10), nullable=False)
    department_name = Column(String(255), nullable=False)
    file_hash = Column(String(64), unique=True, nullable=False)
    created_at = Column(DateTime, server_default=func.now())
```

Сессии SQLAlchemy управляются через контекстный менеджер: при каждом запросе создаётся новая сессия, которая автоматически закрывается после окончания обработки. Это гарантирует, что соединения с базой данных не «утекают» и корректно возвращаются в пул.

Все запросы к базе данных используют параметризацию (через ORM-методы `filter` и `filter_by`), что полностью исключает возможность SQL-инъекций. При сохранении документа с успешно пройденной проверкой хеш файла проверяется на уникальность; если запись с таким хешем уже существует, операция вставки не выполняется, и пользователь получает соответствующее уведомление.

3.2.9 Предотвращение дублирования

Для исключения повторной загрузки одинаковых документов используется вычисление хеш-суммы файла (алгоритм SHA-256). Хеш вычисляется блоками по 8192 байта, что позволяет обрабатывать файлы любого размера без полной загрузки в память.

При успешной проверке (документ не содержит ошибок) хеш сохраняется в базе данных вместе с метаданной: название документа, ФИО автора, код факультета, название кафедры, дата загрузки. При каждой новой загрузке система выполняет следующие действия:

1. Вычисляется хеш загружаемого файла.
2. Выполняется запрос к базе данных на поиск записи с таким же хешем.
3. Если запись найдена, загрузка отклоняется, пользователю возвращается сообщение «Данный документ уже был загружен ранее».
4. Если запись не найдена, документ сохраняется, и в базу добавляется новая запись.

Это предотвращает накопление дубликатов и экономит дисковое пространство.

3.2.10 Обработка ошибок и восстановление

При работе с COM-интерфейсом Microsoft Word возможны различные сбои: документ может быть повреждён, Word может зависнуть или завершиться аварийно. В модуле реализована многоуровневая обработка исключений:

1. На уровне COM-вызовов — каждый вызов метода Word обернут в блок try-except. При возникновении ошибки (например, попытка открыть повреждённый файл) выполняется корректное закрытие документа и завершение процесса Word через `self.word.Quit(False)`. Пользователь получает сообщение с указанием причины ошибки и предложением проверить файл.

2. На уровне проверки документа — при сбое на любом этапе (фильтрация абзацев, извлечение параметров, группировка ошибок) проверка прерывается, пользователю возвращается код ошибки 500 Internal Server Error с пояснением. Временные файлы удаляются в блоке finally, чтобы не засорять дисковое пространство.

3. На уровне базы данных — при нарушении уникальности (попытка вставить дубликат) или других ограничениях целостности формируется понятное сообщение об ошибке без раскрытия деталей SQL-запроса.

Дополнительно реализован механизм тайм-аутов: если проверка документа занимает более 600 секунд (10 минут), операция прерывается, и пользователь получает сообщение о необходимости уменьшить размер файла или разбить его на части.

3.2.11 Хранение данных

Для хранения метаданных используется таблица `manual` в PostgreSQL, содержащая следующие поля:

- `id_manual` — первичный ключ (автоинкремент);
- `manual_name` — название документа;
- `fio_user` — ФИО автора;
- `faculty_code` — код факультета;

- department_name — название кафедры;
- file_hash — уникальный хеш (SHA-256);
- created_at — дата и время загрузки.

Сами файлы хранятся в файловой системе в директориях checked_files/ (проверенные документы с выделенными ошибками) и reports/ (PDF-отчёты). В базе хранятся только пути к этим файлам. Такой подход снижает нагрузку на СУБД и позволяет хранить файлы любого размера без ограничений.

3.2.12 Логирование

Все операции модуля логируются с использованием встроенного модуля logging. Настроены два уровня логирования:

1. INFO — фиксируются штатные операции: загрузка файла, успешное завершение проверки, генерация титульного листа, отправка результата пользователю, авторизация и выход пользователя.

2. ERROR — фиксируются сбои: невозможность открыть документ через COM-интерфейс, ошибка при сохранении файла, проблема с подключением к базе данных, тайм-аут проверки.

Логи сохраняются в файл с ротацией по дате, что позволяет администратору системы анализировать причины отказов, отслеживать подозрительную активность и своевременно устранять неполадки.

3.2.13 Заключение

Разработанный модуль обработки учебно-методических документов реализует полный цикл работы с документами формата DOCX: приём файлов, автоматическую проверку оформления через COM-интерфейс Microsoft Word, группировку ошибок, формирование исправленной копии с визуальной разметкой, генерацию PDF-отчётов, создание титульных листов через python-docx (с различными шаблонами для методических указаний, учебных пособий и монографий), управление сессиями через двухуровневую систему токенов, предотвращение дублирования с помощью хеширования, надёжное хранение метаданных в PostgreSQL, а также многоуровневую обработку

ошибок и логирование. Все компоненты модуля спроектированы с учётом требований к производительности, расширяемости и отказоустойчивости.

3.3 Проект базы данных

В качестве системы управления базами данных выбрана PostgreSQL. Эта СУБД обеспечивает высокую надёжность хранения информации, полную поддержку транзакций, соблюдение принципов ACID, а также располагает встроенными механизмами для контроля целостности данных на уровне ограничений и ссылочной целостности.

Подключение к базе данных реализовано через отдельный модуль `database.py`, который отвечает за конфигурацию параметров соединения, инициализацию подключений и их корректное завершение. В приложении задействован пул соединений — это позволяет переиспользовать уже установленные подключения, снижая накладные расходы на установку нового соединения для каждого запроса, и эффективно обслуживать большое количество одновременно работающих пользователей.

Для описания структур данных применяется объектно-реляционное отображение (ORM) SQLAlchemy. Каждая сущность предметной области представлена в коде отдельным классом, который описывает схему соответствующей таблицы: перечень полей, их типы, ограничения, а также связи с другими таблицами (один-ко-многим, многие-ко-многим). Такой подход упрощает взаимодействие с базой данных и снижает вероятность ошибок при формировании SQL-запросов.

3.3.1 Схема базы данных

На основе анализа предметной области были выделены основные сущности системы и связи между ними. Схема базы данных отражает структуру данных и взаимосвязи между пользователями, кафедрами, факультетами и загруженными документами.

Схема базы данных представлена на рисунке 3.2.

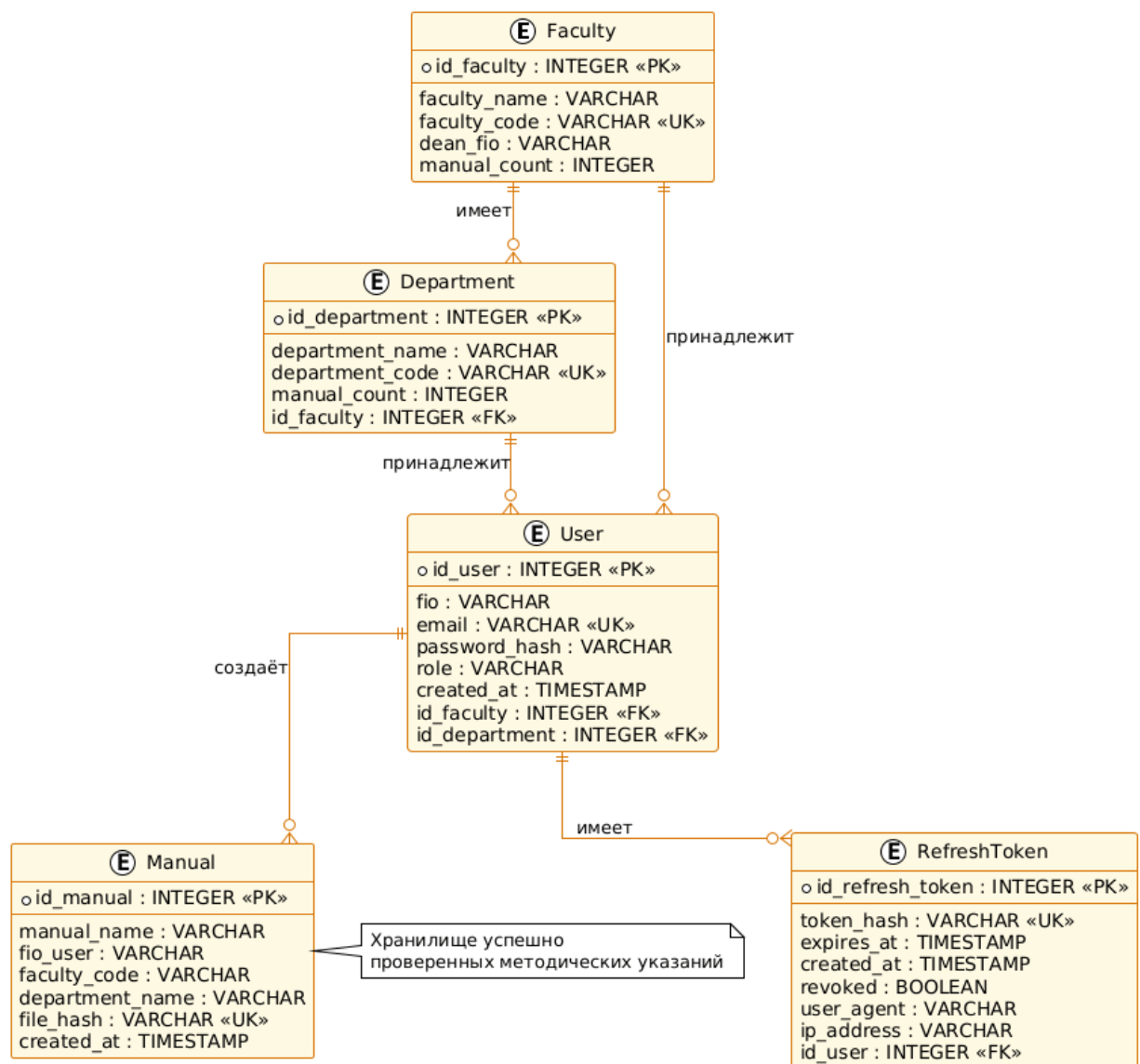


Рисунок 3.2 – Схема базы данных

В модели центральное место занимает сущность «manual», связанная с пользователями и организационной структурой (кафедры и факультеты). Также выделена сущность токенов обновления, обеспечивающая механизм аутентификации пользователей.

К числу основных сущностей базы данных относятся:

- пользователи (users) — содержат учётные записи лиц, работающих с системой (авторы, преподаватели, сотрудники);
- факультеты (faculty) — подразделения университета, к которым привязаны пользователи и кафедры;
- кафедры (department) — структурные единицы внутри факультетов;

- документы (manual) — таблица, хранящая сведения о всех типах учебно-методических материалов (методические указания, учебные пособия, монографии), успешно прошедших проверку;
- токены обновления (refresh_tokens) — используются для поддержания сеансов авторизации.

3.3.2 Нормализация

Структура базы данных спроектирована в соответствии с требованиями третьей нормальной формы.

Каждая таблица оснащена первичным ключом, все поля содержат неделимые значения, а взаимосвязи между сущностями организованы через механизм внешних ключей. Избыточность данных устранена путём декомпозиции информации на логически независимые таблицы (факультеты, кафедры, пользователи).

Помимо этого, в схеме предусмотрены дополнительные ограничения, гарантирующие целостность данных. К ним относятся уникальные индексы (например, для адреса электронной почты пользователя и хеш-суммы файла документа), а также проверочные условия (например, допустимые значения для роли пользователя).

3.3.3 Реляционная модель данных

На основе схемы базы данных построена реляционная модель базы данных, отражающая структуру таблиц и их взаимосвязи.

Схема реляционной модели представлена на рисунке 3.3.

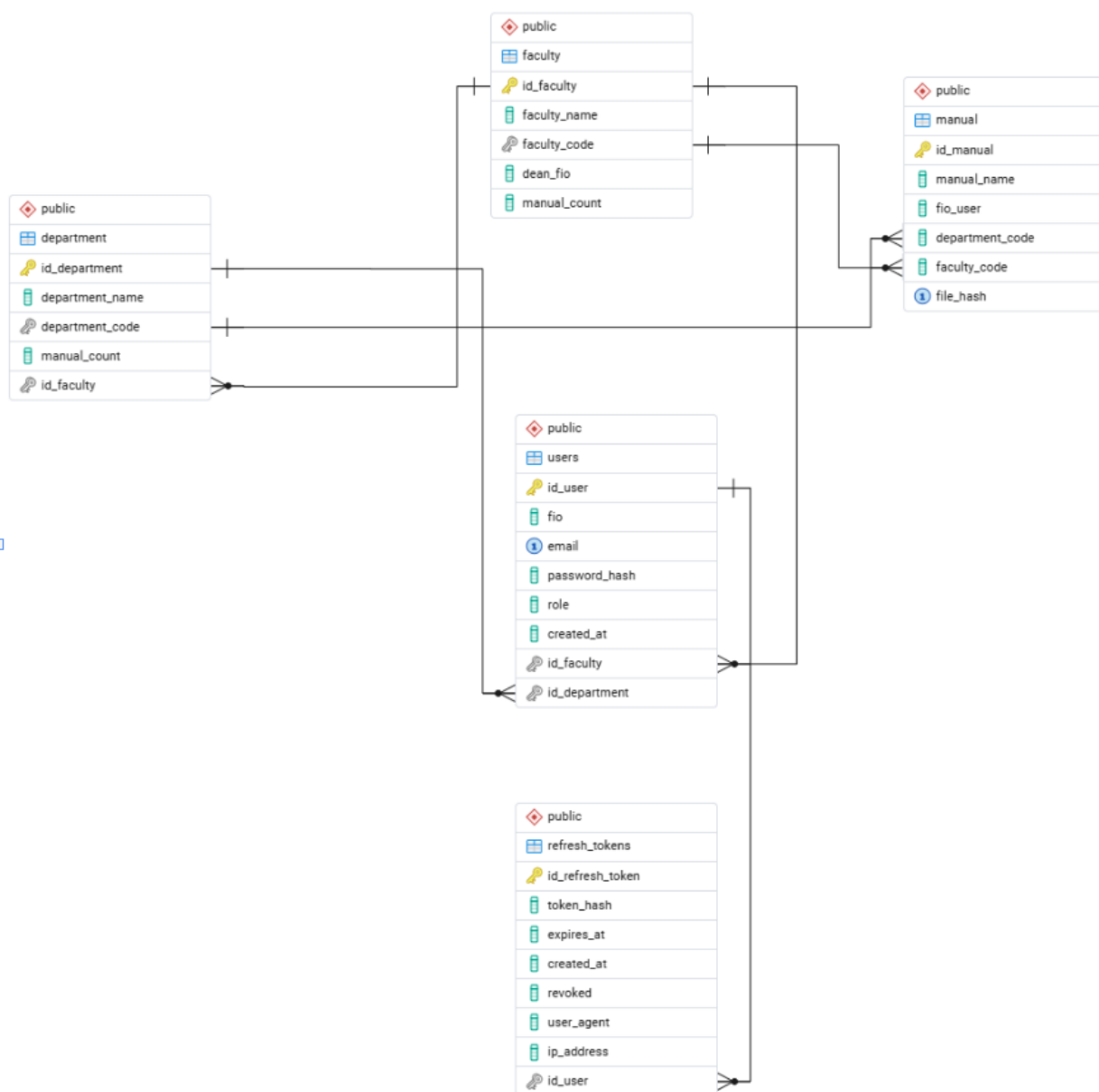


Рисунок 3.3 – Реляционная модель базы данных

Реляционная модель включает таблицы пользователей, факультетов, кафедр, документов (методических указаний, учебных пособий, монографий) и токенов обновления. Связи между таблицами реализованы через внешние ключи, что обеспечивает целостность данных и согласованность структуры.

Особое значение имеет таблица «manual», используемая для хранения информации о всех загружаемых документах и предотвращения их дублирования.

3.3.4 Описание структуры базы данных

Центральным элементом базы данных выступает таблица «manual». Она служит для хранения сведений о загружаемых документах, причём тип документа значения не имеет. В каждой записи фиксируются: название документа, ФИО автора (фамилия, имя, отчество), код факультета, название кафедры и уникальная хеш-сумма файла. Хеш рассчитывается исходя из содержимого документа и используется для защиты от дублирования — повторная загрузка того же файла не приведёт к появлению ещё одной записи. Это исключает дублирование и экономит дисковое пространство.

Таблицы факультетов и кафедр обеспечивают связь пользователей с организационной структурой. Каждый пользователь закреплён за конкретным факультетом и кафедрой, что позволяет определить подразделение автора документа и, при необходимости, сформировать статистику по кафедрам или факультетам.

Документы в виде файлов размещаются непосредственно в файловой системе сервера. Это упрощает работу с большими объёмами данных и обеспечивает быстрый доступ к содержимому. В базе данных при этом сохраняются только метаданные (название, автор, дата загрузки) и уникальные идентификаторы, необходимые для поиска файла в хранилище. Благодаря такому разделению снижается нагрузка на СУБД, а система легко масштабируется при увеличении количества загружаемых документов.

Детальное описание структуры всех таблиц базы данных представлено в таблицах 3.1–3.5.

Таблица 3.1 – Атрибуты таблицы «faculty» (Факультеты)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_faculty	SERIAL	
	*	faculty_name	VARCHAR	255
UK	*	faculty_code	INTEGER	
	*	dean_fio	VARCHAR	255

Продолжение таблицы 3.1

Key Type	Optionality	Column name	Data Type	Size
		manual_count	INTEGER	

Таблица 3.2 – Атрибуты таблицы «department» (Кафедры)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_department	SERIAL	
	*	department_name	VARCHAR	255
UK	*	department_code	INTEGER	
		manual_count	INTEGER	
FK	*	id_faculty	INTEGER	

Таблица 3.3 – Атрибуты таблицы «users» (Пользователи)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_user	SERIAL	
	*	fio	VARCHAR	255
UK	*	email	VARCHAR	255
	*	password_hash	VARCHAR	255
	*	role	VARCHAR	50
		created_at	TIMESTAMP	
FK	*	id_faculty	INTEGER	
FK	*	id_department	INTEGER	

Таблица 3.4 – Атрибуты таблицы «manual» (Документы)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_manual	SERIAL	
	*	manual_name	VARCHAR	255
	*	fio_user	VARCHAR	255

Продолжение таблицы 3.4

Key Type	Optionality	Column name	Data Type	Size
	*	department_code	INTEGER	
	*	faculty_code	INTEGER	
UK	*	file_hash	VARCHAR	64

Таблица 3.5 – Атрибуты таблицы «refresh_tokens» (Токены обновления)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_refresh_token	SERIAL	
UK	*	token_hash	VARCHAR	255
	*	expires_at	TIMESTAMP	
		created_at	TIMESTAMP	
	*	revoked	BOOLEAN	
		user_agent	VARCHAR	500
		ip_address	VARCHAR	100
FK	*	id_user	INTEGER	

4 Рабочий проект

4.1 Спецификация компонентов программной системы

В данном разделе приведено описание основных модулей и классов, разработанных в процессе реализации серверной части информационно-аналитической системы. Для каждого класса указаны его назначение, а также перечень методов с модификаторами доступа и кратким описанием. Для модулей, состоящих из набора функций, приводится описание каждой функции.

4.1.1 Класс WordMethodicalChecker (модуль checker.py)

Класс WordMethodicalChecker является центральным компонентом модуля проверки документов. Он реализует алгоритм автоматической проверки оформления DOCX-файлов через COM-интерфейс Microsoft Word. Класс обеспечивает открытие документа, анализ параметров форматирования (шрифт, размер, отступы, интервалы), проверку полей страницы и нумерации, а также формирование списка ошибок. Методы класса перечислены в таблице 4.1.

Таблица 4.1 – Методы класса WordMethodicalChecker (checker.py)

Название	Доступ	Описание
__init__	public	конструктор, инициализирует параметры проверки (видимость Word, необходимость создания исправленной копии).
__enter__	public	запуск экземпляра Microsoft Word в фоновом режиме, отключение всплывающих окон.
__exit__	public	корректное завершение процесса Word.

Продолжение таблицы 4.1

Название	Доступ	Описание
open_document	public	открытие DOCX-файла для чтения или с правом записи.
check	public	запуск полной проверки документа: анализ абзацев, полей страницы, нумерации, возврат отчёта и пути к исправленному файлу.
_check_paragraphs	private	перебор абзацев, извлечение параметров форматирования, фиксация ошибок шрифта, размера, отступа, интервала.
_check_margins	private	проверка полей страницы (верхнее, нижнее, левое, правое) на соответствие эталонам.
_check_page_numbers	private	проверка наличия и расположения нумерации страниц.
_group_pages	private	группировка последовательных страниц с однотипными ошибками в диапазоны.

4.1.2 Класс CheckReport (модуль checker.py)

Класс CheckReport представляет собой структуру для хранения результатов проверки документа. Он содержит список выявленных ошибок и предоставляет метод для форматирования отчёта в текстовый вид. Методы класса приведены в таблице 4.2.

Таблица 4.2 – Методы класса CheckReport (checker.py)

Название	Доступ	Описание
<code>__init__</code>	public	инициализация пустого списка ошибок.
<code>to_text</code>	public	формирование текстового представления отчёта для отображения пользователю.

4.1.3 Модуль генерации титульных листов (titlePageGenerator.py)

Модуль titlePageGenerator.py содержит функции для программного создания DOCX-файлов титульных листов методических указаний, учебных пособий и монографий с использованием библиотеки python-docx. Функции модуля перечислены в таблице 4.3.

Таблица 4.3 – Функции модуля titlePageGenerator.py

Название	Доступ	Описание
<code>generate_title_page_docx</code>	public	генерация титульного листа для методических указаний (название, дисциплина, направление, составитель, рецензенты, УДК, описание).
<code>generate_tutorial_title_page_docx</code>	public	генерация титульного листа для учебного пособия (автор, название, рецензенты, направления, значение А, ISBN, УДК, ББК, описание).

Продолжение таблицы 4.3

Название	Доступ	Описание
generate_monograph_title_page_docx	public	генерация титульного листа для монографии (авторы в одну строку, название, город, год, УДК, ББК, ISBN, описание).
add_paragraph	private	вспомогательная функция для добавления абзаца с заданными параметрами форматирования.
add_empty_paragraphs	private	добавление пустых абзацев для вертикального отступа.
format_author_name_for_biblio	private	преобразование ФИО автора в формат «И.О. Фамилия» для библиографической записи.

4.1.4 Модуль формирования PDF-отчётов (pdfReport.py)

Модуль pdfReport.py содержит функцию для создания структурированного PDF-отчёта по результатам проверки документа с использованием библиотеки reportlab. Функция модуля описана в таблице 4.4.

Таблица 4.4 – Функции модуля pdfReport.py

Название	Доступ	Описание
generate_error_report_pdf	public	формирование PDF-отчёта, содержащего таблицу ошибок (номер, раздел, тип, местоположение, описание) и итоговый статус проверки.

Продолжение таблицы 4.4

Название	Доступ	Описание
register_fonts	private	регистрация шрифтов семейства Times New Roman для корректного отображения кириллицы.

4.1.5 Модуль отправки в РИО (email.py)

Модуль email.py содержит функции для отправки электронных писем с вложениями через SMTP-сервер. Функции модуля перечислены в таблице 4.5.

Таблица 4.5 – Функции модуля email.py

Название	Доступ	Описание
send_email_with_attachments	public	формирование MIME-письма, добавление вложений, отправка через SMTP-сервер.
send_to_rio	public	подготовка письма для РИО с тремя вложениями и комментарием пользователя.

4.1.6 Контроллеры REST API (routers)

В таблицах 4.6 – 4.8 приведено описание методов контроллеров, реализующих маршруты REST API.

Таблица 4.6 – Методы контроллера CheckerController (checker.py)

Название	Доступ	Описание
check_document	public	приём DOCX-файла, запуск проверки, возврат JSON с результатами и ссылкой на скачивание.
get_my_documents	public	возврат списка успешно проверенных документов текущего пользователя.
download_checked_file	public	скачивание DOCX с визуально выделенными ошибками.
open_pdf_report	public	скачивание PDF-отчёта.

Таблица 4.7 – Методы контроллера TitlePageController (title_page.py)

Название	Доступ	Описание
generate_title_page	public	генерация титульного листа для методических указаний по данным из формы.
generate_tutorial_title_page	public	генерация титульного листа для учебного пособия.
generate_monograph_title_page	public	генерация титульного листа для монографии.

Таблица 4.8 – Методы контроллера RioController (rio.py)

Название	Доступ	Описание
upload_step1	public	приём двух вспомогательных файлов (произвольные документы), временное сохранение.

Продолжение таблицы 4.8

Название	Доступ	Описание
upload_step2	public	приём основного DOCX-файла, запуск проверки через WordMethodicalChecker. При ошибках блокирует переход к шагу 3.
submit_step3	public	формирование email с вложениями (три файла), отправка через SMTP, очистка временных файлов.
reset_session	public	сброс сессии, удаление всех временных файлов пользователя.

4.2 Системное тестирование

Системное тестирование информационно-аналитической системы проводилось в соответствии со сценариями использования, определёнными в техническом задании. В процессе тестирования проверялись основные функции системы: автоматическая проверка документов, генерация титульных листов, отправка в РИО, а также корректность обработки ошибок.

Тестирование выполнялось на операционной системе Windows 10 с использованием браузера Google Chrome. Корректность работы системы подтверждена результатами выполнения тест-кейсов.

4.2.1 Сводная таблица тест-кейсов

В таблице 4.9 приведён перечень всех выполненных тест-кейсов с указанием их идентификаторов и результатов.

Таблица 4.9 – Сводная таблица тест-кейсов

ID	Название тест-кейса	Результат
ТС-01	Проверка корректного DOCX-документа	Пройден
ТС-02	Проверка документа с ошибками оформления	Пройден

Продолжение таблицы 4.9

ID	Название тест-кейса	Результат
ТС-03	Генерация титульного листа методических указаний	Пройден
ТС-04	Генерация титульного листа учебного пособия	Пройден
ТС-05	Генерация титульного листа монографии	Пройден
ТС-06	Отправка документов в РИО (полный цикл)	Пройден
ТС-07	Отправка в РИО с ошибкой на шаге 2	Пройден
ТС-08	Скачивание исправленного документа	Пройден
ТС-09	Скачивание PDF-отчёта	Пройден

4.2.2 ТС-01. Проверка корректного DOCX-документа

Предусловие: сервер запущен, пользователь авторизован.

Шаги:

1. Отправить POST-запрос на /checker/check с корректным DOCX-файлом и параметром doc_type>manual.
2. Дождаться завершения проверки.

Ожидаемый результат: в ответе поле has_errors: false, документ сохраняется в базе данных. PDF-отчёт не формируется.

Фактический результат: пройден.

4.2.3 ТС-02. Проверка документа с ошибками оформления

Предусловие: сервер запущен, пользователь авторизован.

Шаги:

1. Отправить POST-запрос на /checker/check с DOCX-файлом, содержащим нарушения оформления (неверный шрифт, отсутствие отступа).
2. Дождаться завершения проверки.

Ожидаемый результат: в ответе поле has_errors: true. Формируются:
– исправленная копия DOCX с выделением ошибок красным цветом;

- PDF-отчёт с таблицей ошибок (номер, тип, местоположение, описание);
- ссылки на оба файла возвращаются клиенту.

Фактический результат: пройден.

4.2.4 ТС-03. Генерация титульного листа методических указаний

Предусловие: сервер запущен, пользователь авторизован.

Шаги:

1. Отправить POST-запрос на /title-page/generate с JSON, содержащим manual_title, discipline_name, direction_code, compiler_name, reviewers (массив), udk, description.

2. Получить ответ.

Ожидаемый результат: библиотека python-docx формирует DOCX-файл с двумя страницами титульного листа. Файл возвращается клиенту.

Фактический результат: пройден.

4.2.5 ТС-04. Генерация титульного листа учебного пособия

Предусловие: сервер запущен, пользователь авторизован.

Шаги:

1. Отправить POST-запрос на /title-page/generate-tutorial с JSON, содержащим author_name, tutorial_title, city, year, reviewers (массив), a_value, isbn, directions (массив), udk, bbk, description.

2. Получить ответ.

Ожидаемый результат: формируется DOCX-файл с правильно оформленным титульным листом учебного пособия.

Фактический результат: пройден.

4.2.6 ТС-05. Генерация титульного листа монографии

Предусловие: сервер запущен, пользователь авторизован.

Шаги:

1. Отправить POST-запрос на /title-page/generate-monograph с JSON, содержащим authors (массив), monograph_title, city, year, udk, bbk, isbn, description.

2. Получить ответ.

Ожидаемый результат: формируется DOCX-файл с авторами в одну строку, названием, выходными данными и аннотацией.

Фактический результат: пройден.

4.2.7 ТС-06. Отправка документов в РИО (полный цикл)

Предусловие: сервер запущен, пользователь авторизован.

Шаги:

1. Отправить POST-запрос на /rio/upload-step1 с двумя вспомогательными файлами (PDF, TXT).

2. Отправить POST-запрос на /rio/upload-step2 с основным DOCX-файлом.

3. Отправить POST-запрос на /rio/submit-step3 с JSON-полем comment (до 1000 символов).

Ожидаемый результат: система отображает сообщение об успешной отправке. На почту РИО приходит письмо с тремя вложениями.

Фактический результат: пройден.

4.2.8 ТС-07. Отправка в РИО с ошибкой на шаге 2

Предусловие: сервер запущен, пользователь авторизован.

Шаги:

1. Отправить POST-запрос на /rio/upload-step1 с двумя вспомогательными файлами.

2. Отправить POST-запрос на /rio/upload-step2 с DOCX-файлом, содержащим ошибки оформления.

Ожидаемый результат: сервер возвращает HTTP-статус 422 и сообщение «В данном документе «имя_файла» имеются ошибки, выполните полную проверку файла». Переход к шагу 3 блокируется.

Фактический результат: пройден.

4.2.9 ТС-08. Скачивание исправленного документа

Предусловие: ранее выполнен успешный ТС-02, получена ссылка на исправленный файл.

Шаги:

1. Перейти по ссылке /checker/download-checked/{filename}.

Ожидаемый результат: браузер скачивает DOCX-файл с выделенными ошибками (красный цвет, комментарии).

Фактический результат: пройден.

4.2.10 ТС-09. Скачивание PDF-отчёта

Предусловие: ранее выполнен успешный ТС-02, получена ссылка на PDF-отчёт.

Шаги:

1. Перейти по ссылке /checker/report/{filename}.

Ожидаемый результат: браузер скачивает PDF-файл с таблицей ошибок и итоговым статусом проверки.

Фактический результат: пройден.

4.2.11 Нефункциональное тестирование

Помимо функциональных тестов, были проверены нефункциональные требования, описанные в техническом задании.

Быстродействие: проверка DOCX-документов объёмом до 100 страниц выполняется не более чем за 120 секунд. Генерация титульного листа занимает не более 5 секунд. Формирование PDF-отчёта завершается менее чем за 3 секунды.

Совместимость с браузерами: клиентская часть протестирована в браузерах Google Chrome, Mozilla Firefox, Yandex. Во всех браузерах корректно работают загрузка файлов, drag-and-drop, динамические формы и скачивание файлов.

Надёжность: при разрыве соединения с базой данных сервер возвращает HTTP-статус 500. При ошибках COM-интерфейса Word система корректно завершает процесс и очищает временные файлы. Транзакционное сохранение проверенных документов гарантирует целостность данных.

Безопасность: доступ к защищённым маршрутам предоставляется только при наличии валидного JWT-токена. Пароли пользователей хранятся в хешированном виде (Argon2). Административные функции доступны только пользователям с соответствующей ролью.

Все нефункциональные требования выполнены в полном объёме.

4.2.12 Результаты тестирования

В ходе системного тестирования были успешно проверены все функциональные и нефункциональные требования, предъявленные к информационно-аналитической системе. Разработанное приложение работает стабильно, интерфейс интуитивно понятен, все предусмотренные сценарии использования выполняются корректно. Система готова к промышленной эксплуатации в редакционно-издательском отделе вуза.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была разработана информационно-аналитическая система автоматизации проверки учебно-методических материалов редакционно-издательского отдела. Разработанная система позволяет выполнять автоматизированную проверку документов на соответствие требованиям оформления, формировать отчёты о выявленных нарушениях, генерировать титульные листы и обеспечивать централизованное хранение данных.

В ходе работы был проведён анализ предметной области, связанной с организацией процессов проверки и сопровождения учебно-методической документации. Особое внимание было уделено проблемам ручной проверки документов, требованиям к оформлению учебных материалов и существующим подходам к автоматизации документооборота. Выполненный анализ позволил сформировать требования к функциональности разрабатываемой системы.

Разработанная система реализует следующие основные возможности:

- аутентификацию и авторизацию пользователей;
- загрузку и обработку учебно-методических документов;
- автоматическую проверку параметров форматирования;
- формирование PDF-отчётов по результатам проверки;
- генерацию титульных листов документов;
- взаимодействие пользователей с редакционно-издательским отделом через веб-интерфейс.

Для реализации системы использована клиент-серверная архитектура. Серверная часть разработана с использованием FastAPI и PostgreSQL, а клиентская часть реализована с применением HTML, CSS и JavaScript. Для хранения и обработки данных использована технология ORM SQLAlchemy.

В процессе разработки было проведено тестирование системы, включающее проверку корректности обработки документов, формирования отчётов, работы механизмов аутентификации и взаимодействия клиентской и сервер-

ной части. Результаты тестирования подтвердили корректность функционирования разработанной системы и соответствие реализованного функционала поставленным требованиям.

Основные результаты выпускной квалификационной работы:

1. Проведён анализ процессов проверки учебно-методических материалов и существующих средств автоматизации документооборота.
2. Разработана архитектура информационно-аналитической системы.
3. Реализована серверная часть системы с использованием FastAPI и PostgreSQL.
4. Разработана клиентская часть веб-приложения.
5. Реализованы модули проверки документов, генерации титульных листов и формирования PDF-отчётов.
6. Проведено тестирование системы и подтверждена корректность её работы.

Все задачи, поставленные в начале выпускной квалификационной работы, были успешно решены. Таким образом, цель выпускной квалификационной работы — разработка информационно-аналитической системы автоматизации проверки учебно-методических материалов редакционно-издательского отдела — была достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лутц, М. Изучаем Python. Том 1 / М. Лутц. – 5-е изд. – Санкт-Петербург : Питер, 2019. – 832 с. – ISBN 978-5-907144-52-1. – Текст : непосредственный.
2. Рамальо, Л. Python. К вершинам мастерства / Л. Рамальо. – Москва : ДМК Пресс, 2017. – 768 с. – ISBN 978-5-97060-384-0. – Текст : непосредственный.
3. Python Documentation : официальный сайт. – URL: <https://docs.python.org/3/> (дата обращения: 10.01.2026).
4. PEP 8 – Style Guide for Python Code : официальный сайт. – URL: <https://peps.python.org/pep-0008/> (дата обращения: 15.01.2026).
5. Threading — Thread-based parallelism // Python Documentation : официальный сайт. – URL: <https://docs.python.org/3/library/threading.html> (дата обращения: 20.01.2026).
6. Logging HOWTO // Python Documentation : официальный сайт. – URL: <https://docs.python.org/3/howto/logging.html> (дата обращения: 25.01.2026).
7. Любанович, Б. FastAPI: веб-разработка на Python / Б. Любанович. – Санкт-Петербург : Питер, 2024. – 288 с. – ISBN 978-601-08-3847-5. – Текст : непосредственный.
8. FastAPI Documentation : официальный сайт. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 01.02.2026).
9. Pydantic Documentation : официальный сайт. – URL: <https://docs.pydantic.dev/> (дата обращения: 05.02.2026).
10. Uvicorn Documentation : официальный сайт. – URL: <https://www.uvicorn.org/> (дата обращения: 10.02.2026).
11. Starlette Documentation : официальный сайт. – URL: <https://www.starlette.io/> (дата обращения: 14.02.2026).
12. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – 8-е изд. – Москва : Вильямс, 2005. – 1328 с. – ISBN 978-5-8459-0788-2. – Текст : непосредственный.

13. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. – 3-е изд. – Москва : Вильямс, 2003. – 1440 с. – ISBN 978-5-8459-0527-3. – Текст : непосредственный.
14. Грофф, Дж. Р. SQL: полное руководство / Дж. Р. Грофф, П. Н. Вайнберг, Э. Дж. Оппель. – 3-е изд. – Санкт-Петербург : Диалектика, 2020. – 960 с. – ISBN 978-5-907114-26-5. – Текст : непосредственный.
15. Моргунов, Е. П. PostgreSQL. Основы языка SQL / Е. П. Моргунов. – Санкт-Петербург : БХВ-Петербург, 2024. – 336 с. – ISBN 978-5-9775-4022-3. – Текст : непосредственный.
16. PostgreSQL Documentation : официальный сайт. – URL: <https://www.postgresql.org/docs/> (дата обращения: 18.02.2026).
17. SQLAlchemy Documentation : официальный сайт. – URL: <https://docs.sqlalchemy.org/> (дата обращения: 22.02.2026).
18. Alembic – Database Migrations for SQLAlchemy : официальный сайт. – URL: <https://alembic.sqlalchemy.org/> (дата обращения: 26.02.2026).
19. Psycopg – PostgreSQL adapter for Python : официальный сайт. – URL: <https://www.psycopg.org/docs/> (дата обращения: 01.03.2026).
20. python-docx Documentation : официальный сайт. – URL: <https://python-docx.readthedocs.io/> (дата обращения: 05.03.2026).
21. pywin32 Documentation // GitHub : официальный сайт. – URL: <https://github.com/mhammond/pywin32> (дата обращения: 09.03.2026).
22. ECMA-376 Office Open XML File Formats : официальный сайт. – URL: <https://www.ecma-international.org/publications-and-standards/standards/ecma-376/> (дата обращения: 13.03.2026).
23. JWT Introduction : официальный сайт. – URL: <https://jwt.io/introduction> (дата обращения: 17.03.2026).
24. The OAuth 2.0 Authorization Framework : официальный сайт. – URL: <https://datatracker.ietf.org/doc/html/rfc6749> (дата обращения: 21.03.2026).
25. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. – Санкт-Петербург : Питер, 2016. – 464 с. – ISBN 978-5-496-00487-9. – Текст : непосредственный.

26. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург : Питер, 2018. – 352 с. – ISBN 978-5-4461-0772-8. – Текст : непосредственный.
27. Рамбо, Дж. UML 2.0. Объектно-ориентированное моделирование и разработка / Дж. Рамбо, М. Блаха. – 2-е изд. – Санкт-Петербург : Питер, 2021. – 542 с. – ISBN 978-5-4461-9428-5. – Текст : непосредственный.
28. Unified Modeling Language Specification, Version 2.5.1 : официальный сайт. – URL: <https://www.omg.org/spec/UML/2.5.1/PDF> (дата обращения: 25.03.2026).
29. draw.io Documentation : официальный сайт. – URL: <https://www.drawio.com/doc/> (дата обращения: 28.03.2026).
30. Чакон, С. Git для профессионального программиста / С. Чакон, Б. Штрауб. – Санкт-Петербург : Питер, 2016. – 496 с. – ISBN 978-5-496-01763-3. – Текст : непосредственный.
31. Git Documentation : официальный сайт. – URL: <https://git-scm.com/doc> (дата обращения: 31.03.2026).
32. Дзекетт, Д. HTML и CSS. Разработка и создание веб-сайтов / Д. Дзекетт. – Москва : Эксмо, 2014. – 480 с. – ISBN 978-5-699-64193-2. – Текст : непосредственный.
33. Дзекетт, Д. JavaScript и jQuery. Интерактивная web-разработка / Д. Дзекетт. – Москва : Эксмо, 2017. – 640 с. – ISBN 978-5-699-80285-2. – Текст : непосредственный.
34. Фрейн, Б. HTML5 и CSS3 и Web 2.0. Разработка современных web-сайтов / Б. Фрейн. – 2-е изд. – Санкт-Петербург : Питер, 2022. – 384 с. – ISBN 978-5-9775-0596-3. – Текст : непосредственный.
35. HTML // MDN Web Docs : официальный сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/HTML> (дата обращения: 02.04.2026).
36. CSS // MDN Web Docs : официальный сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/CSS> (дата обращения: 04.04.2026).
37. JavaScript // MDN Web Docs : официальный сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript> (дата обращения: 06.04.2026).

38. HTTP // MDN Web Docs : официальный сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/HTTP> (дата обращения: 08.04.2026).
39. Fielding, R. HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. – RFC 9110. – URL: <https://www.rfc-editor.org/rfc/rfc9110.html> (дата обращения: 10.04.2026).
40. Fielding, R. Architectural Styles and the Design of Network-based Software Architectures // University of California : официальный сайт. – URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (дата обращения: 12.04.2026).
41. Оккен, Б. pytest. Быстрое тестирование приложений на Python / Б. Оккен. – Санкт-Петербург : Питер, 2020. – 320 с. – ISBN 978-1-6805-0240-4. – Текст : непосредственный.
42. PyCharm Documentation : официальный сайт. – URL: <https://www.jetbrains.com/pycharm/documentation/> (дата обращения: 14.04.2026).
43. Bootstrap Documentation : официальный сайт. – URL: <https://getbootstrap.com/docs/5.3/> (дата обращения: 16.04.2026).
44. Docker Documentation : официальный сайт. – URL: <https://docs.docker.com/> (дата обращения: 18.04.2026).
45. NGINX Documentation : официальный сайт. – URL: <https://nginx.org/en/docs/> (дата обращения: 20.04.2026).
46. ReportLab Documentation : официальный сайт. – URL: <https://docs.reportlab.com/> (дата обращения: 22.04.2026).
47. asyncio Documentation // Python Documentation : официальный сайт. – URL: <https://docs.python.org/3/library/asyncio.html> (дата обращения: 24.04.2026).
48. Марц Н., Уоррен Дж. Большие данные: принципы и практика построения масштабируемых систем обработки данных в реальном времени / Н. Марц, Дж. Уоррен. – Москва : Диалектика, 2018. – 368 с. – ISBN 978-5-8459-2075-1. – Текст : непосредственный.
49. Персиваль, Г. Паттерны разработки на Python: TDD, DDD и событийно-ориентированная архитектура / Г. Персиваль, Б. Грегори. –

Санкт-Петербург : Питер, 2025. – 336 с. – ISBN 978-601-12-5028-3. – Текст : непосредственный.

50. Любанович, Б. Простой Python. Современный стиль программирования / Б. Любанович. – Санкт-Петербург : Питер, 2020. – 592 с. – ISBN 978-5-4461-1639-3. – Текст : непосредственный.

51. Лутц, М. Python. Карманный справочник / М. Лутц. – Москва : Эксмо, 2022. – 318 с. – ISBN 978-5-907114-60-9. – Текст : непосредственный.

52. Python Tutorial // Python Documentation : официальный сайт. – URL: <https://docs.python.org/3/tutorial/> (дата обращения: 26.04.2026).

53. Рамальо, Л. Python. К вершинам мастерства. Том 2 / Л. Рамальо. – Москва : ДМК Пресс, 2023. – 672 с. – ISBN 978-5-97060-885-2. – Текст : непосредственный.

54. Дауни, А. Б. Think Python. Аналитический подход к программированию / А. Б. Дауни. – 3-е изд. – Санкт-Петербург : Питер, 2024. – 320 с. – ISBN 978-1-4493-3072-9. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.8.

Сведения о ВКРБ

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ БАКАЛАВРИАТА

Бизнес-проект “Информационно-аналитическая система для РИО вуза.
Разработка сервиса “Методические указания”

Руководитель ВКРБ
к.т.н, доцент
Аникина Елена Игоревна

Автор ВКРБ
студент группы ПО-22б
Петров Михаил Евгеньевич

ВКРБ 2206019.09.03.04.26.021			
Сведения о ВКРБ			
Аннотация	Формат И.О.	Листов	Итого
Резюме	Листов	Итого	
Перечень	Листов	Итого	
Выпускная квалификационная работа бакалавра			
03/У ПО-22б			

Рисунок А.1 – Сведения о ВКРБ

Цель и задачи разработки

2

Цель настоящей работы - разработка информационно-аналитической системы автоматизации учёта и проверки учебно-методических материалов кафедры, обеспечивающей централизованную обработку документов, автоматическую проверку их оформления и формирование сопроводительной документации.

Для достижения поставленной цели необходимо решить следующие задачи:

- разработать схему серверной части информационно-аналитической системы на базе Python, FastAPI и PostgreSQL, предусмотрев модульное построение приложения и обработку входящих запросов от пользователей;
- создать систему регистрации, аутентификации и управления правами доступа пользователей;
- разработать программные интерфейсы (API) для приёма, сохранения, обработки и управления учебно-методическими документами разных типов;
- настроить автоматическую проверку файлов DOCX на соответствие установленным правилам оформления;
- создать механизм наглядного выделения ошибок форматирования в документе с получением копии DOCX-файла, содержащей отметки о нарушениях;
- разработать блок для автоматического создания титульных листов для различных видов учебно-методических материалов;
- обеспечить автоматическое формирование PDF-отчётов по итогам проверки документов;
- внедрить систему хранения данных о пользователях, кафедрах, факультетах и документах с помощью PostgreSQL;
- предусмотреть обработку исключительных ситуаций и ошибок системы с возвратом корректных HTTP-ответов, ведением журнала событий и обеспечением устойчивости серверной части;
- провести модульное и системное тестирование основных компонентов информационно-аналитической системы, включая проверку корректности обработки документов, генерации отчётов, работы механизмов авторизации и взаимодействия с базой данных.

ВКРБ 2206509.09.03.04.26.010			
Имя работы	Фамилия И.О.	Инициалы	Имя
Разработчик	Лавров И.С.		
Проверенный	Алексеев И.А.		
	Чепелев А.А.		
Цели и задачи разработки		Лист 2	Листов 8
Выпускная квалификационная работа бакалавра		03/19 ПО-228	

Рисунок А.2 – Цели и задачи разработки

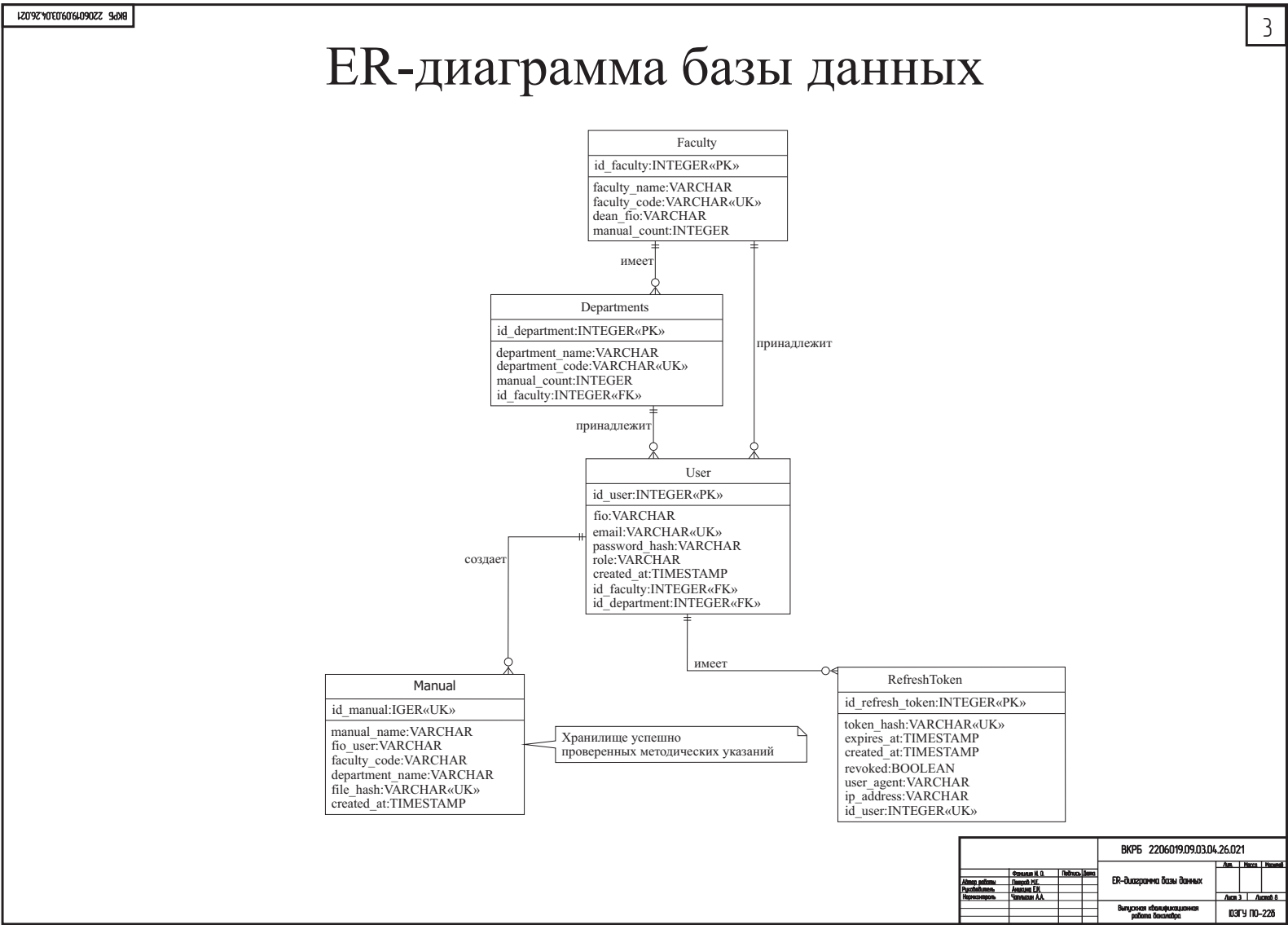


Рисунок А.3 – ER-диаграмма базы данных



Рисунок А.4 – Реляционная модель базы данных

1209290307090304 9d88
5

Страница генерации титульных листов

ю Проверка файлов
Веб-система

- Авторизация
- Профиль
- Проверка
- Титульный лист**
- Статистика
- Отправить в РИО
- Администрирование

Система автономной проверки файлов

Загрузка файлов, проверка оформления, PDF-отчёт и статистика

[Обновить профиль](#)
[Выйти](#)

Генерация титульного листа

Тип титульного листа

Методические указания

Методические указания — первая страница

Название методички

Название дисциплины

Для кого

Студентов

Код направления

09.03.04

Название направления

Город

Курск

Год

2026

Методические указания — вторая страница

УДК

ФИО составителя

Ученая степень, должность рецензента

ФИО рецензента

[Добавить рецензента](#)

Краткое описание

Введите краткое описание

Не более 1000 символов 0 / 1000

[Сгенерировать титульный лист](#)

				ВКРБ 2206019.09.03.04.26.021			
Адрес работы	Фамилия И.О.	Инициалы	Дата	Страница генерации титульных листов	Лист 5	Листов 8	
Полное наименование	Личный ИС	Личный ИС		Высшая квалификационная рабочая документация			
Перечисление	Числовые А.А.			03/У ПО-226			

Рисунок А.5 – Страница генерации титульных листов

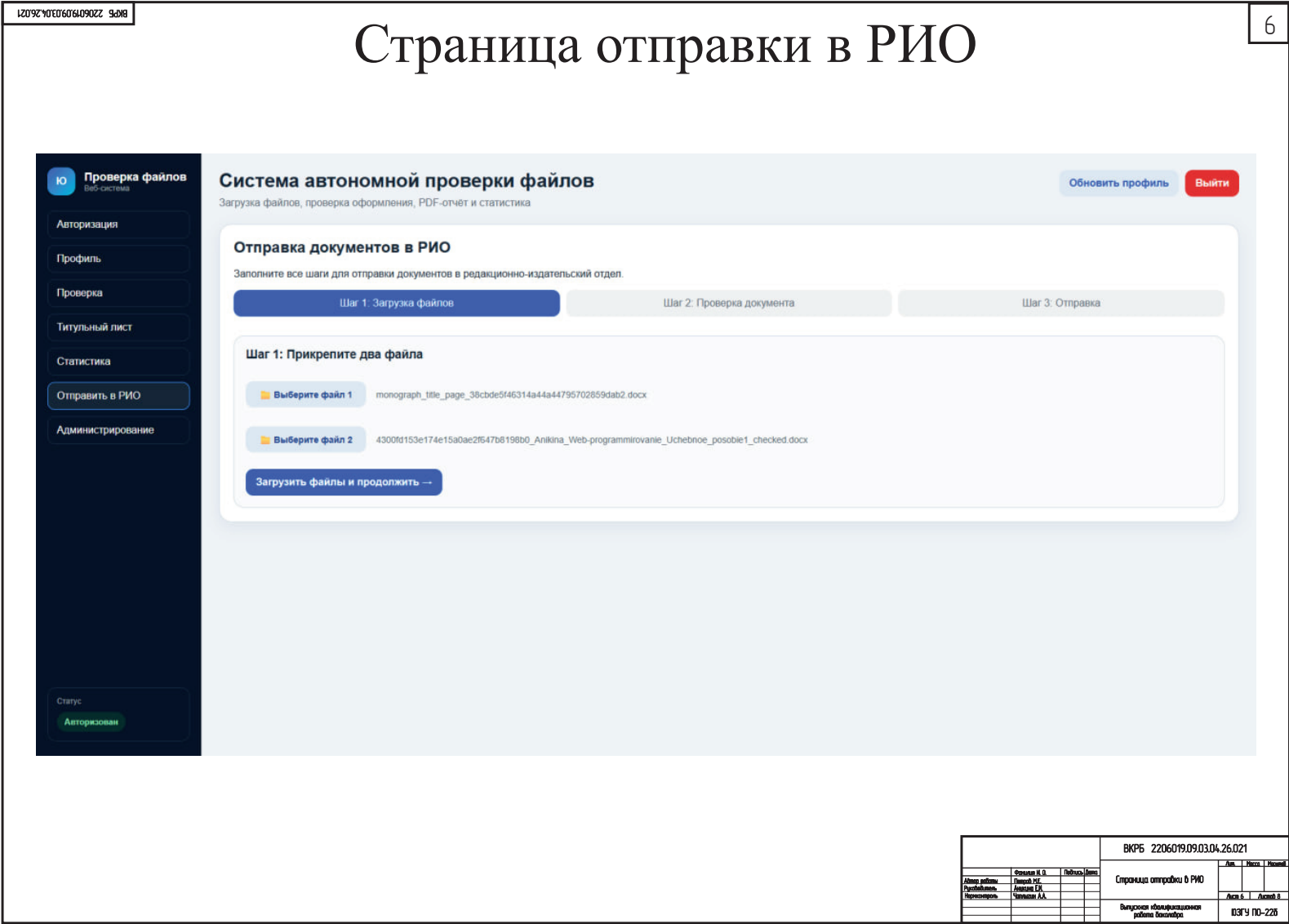


Рисунок А.6 – Страница отправки в РИО

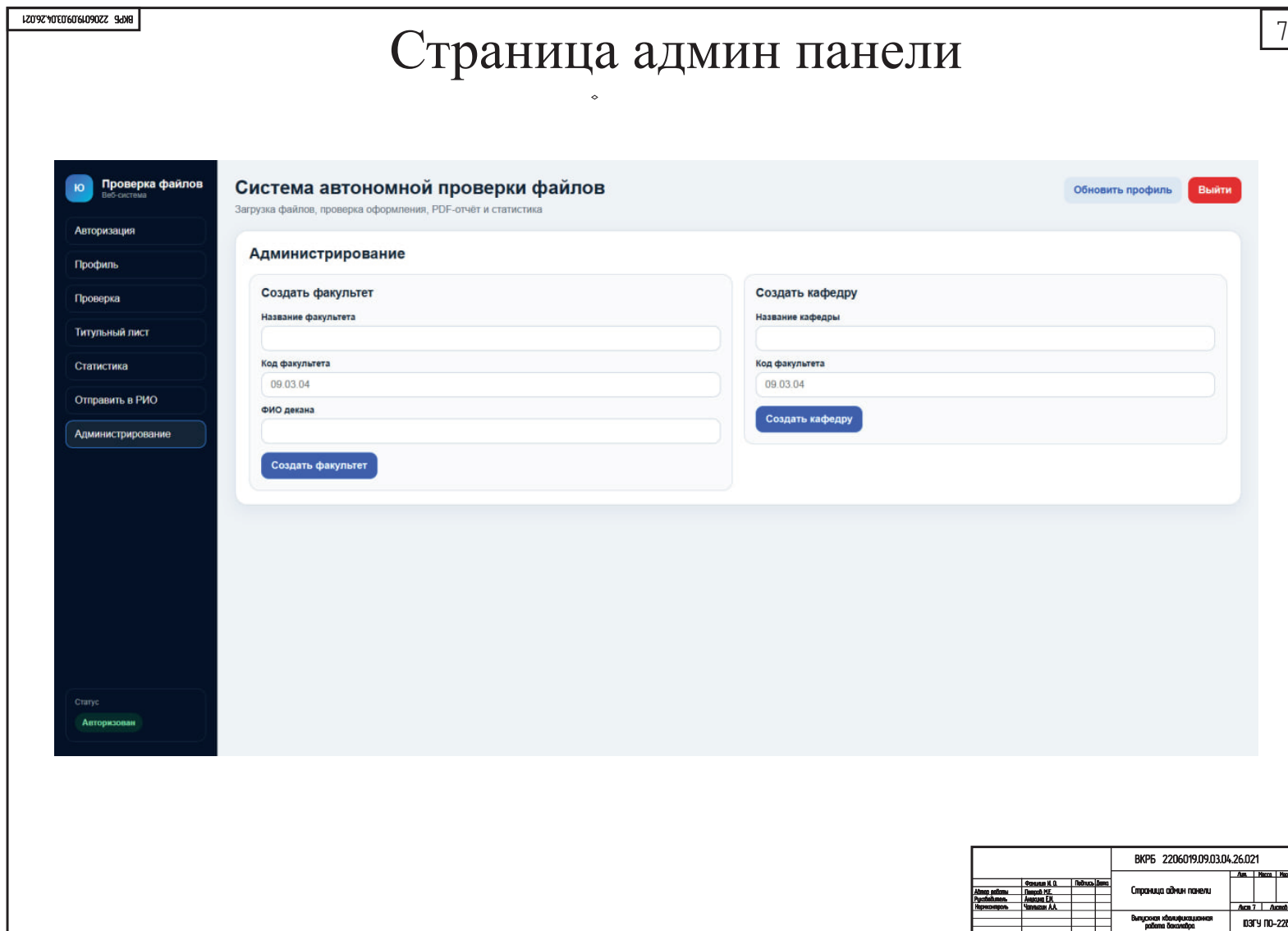


Рисунок А.7 – Страница админ панели

Заключение

В ходе выполнения данной выпускной квалификационной работы создана информационно-аналитическая система для автоматизации учёта и контроля учебно-методических материалов кафедры. Разработанное решение предназначено для автоматизации этапов загрузки, анализа, хранения и обработки методических документов, а также подготовки сопутствующей документации.

Созданная система упрощает деятельность сотрудников кафедры и работников редакционно-издательского отдела. Пользователи могут отправлять файлы на проверку, автоматически анализировать их оформление, создавать титульные страницы и получать PDF-документы с итогами проверки. Администратор системы вправе управлять учётными записями пользователей, факультетами, кафедрами и отслеживать процесс обработки документов.

Основные результаты работы:

1. Проведён анализ предметной области и определены требования к информационно-аналитической системе.
2. Разработана архитектура серверной части системы с использованием Python, FastAPI и PostgreSQL.
3. Реализованы механизмы регистрации, авторизации и разграничения прав доступа пользователей на основе JWT-токенов.
4. Разработан модуль автоматической проверки DOCX-документов на соответствие требованиям оформления, включая проверку шрифтов, полей страницы, заголовков, таблиц и изображений.
5. Реализован механизм визуального выделения ошибок форматирования и формирования PDF-отчётов по результатам проверки документов.
6. Разработан модуль генерации титульных листов для различных типов учебно-методических изданий.
7. Проведено модульное и системное тестирование основных компонентов системы.

Все требования, поставленные в техническом задании, были реализованы, а задачи, сформулированные в начале работы, успешно решены.

				ВКРБ 2206509.09.03.04.26.010			
Имя работы Руководитель, Перепроверил,	Фамилия И.О.	Подпись		Заключение	Акт	Итого	Норматив
	Имя И.О.						
	Имя И.О.				Акт в	Листов в	
Имя работы	Имя И.О.			Выпускная квалификационная работа бакалавра	03/У ПО-228		

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

main.py

```
1 import asyncio
2 import sys
3
4 from fastapi import FastAPI
5 from fastapi.staticfiles import StaticFiles
6 from fastapi.responses import FileResponse
7
8 from app.db.base import Base
9 from app.db.session import engine
10 from app.routers import admin, auth, checker, statistics, title_page, rio
11
12 if sys.platform == 'win32':
13     asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())
14 app = FastAPI(title="Diplom Checker API")
15
16 Base.metadata.create_all(bind=engine)
17
18 app.mount("/static", StaticFiles(directory="app/static"), name="static")
19
20 app.include_router(auth.router)
21 app.include_router(checker.router)
22 app.include_router(admin.router)
23 app.include_router(statistics.router)
24 app.include_router(title_page.router)
25 app.include_router(rio.router)
26
27
28 @app.get("/")
29 def root():
30     return FileResponse("app/templates/index.html")
```

checker.py

```
1 import os
2 import shutil
3 from dataclasses import dataclass, field
4 from typing import List
5
6 try:
7     import win32com.client
8     from win32com.client import dynamic
9 except Exception as e:
10     win32com = None
11     dynamic = None
12     WIN32_IMPORT_ERROR = e
13 else:
14     WIN32_IMPORT_ERROR = None
15
16 CM_TO_PT = 28.3464567
```

```

17
18 WD_ALIGN_CENTER = 1
19 WD_WITHIN_TABLE = 12
20 WD_HEADER_FOOTER_PRIMARY = 1
21 WD_LINE_SPACE_SINGLE = 0
22 WD_STATISTIC_PAGES = 2
23 WD_ACTIVE_END_PAGE_NUMBER = 3
24
25 WD_COLOR_RED = 255
26 WD_FIELD_PAGE = 33
27
28
29 def pt_to_cm(pt):
30     return pt / CM_TO_PT
31
32 @dataclass
33 class CheckIssue:
34     rule: str
35     severity: str
36     location: str
37     message: str
38     priority: int = 3
39
40
41 @dataclass
42 class CheckReport:
43     issues: List[CheckIssue] = field(default_factory=list)
44
45     def to_text(self):
46         if not self.issues:
47             return "❑ Ошибок не найдено."
48
49         self.issues.sort(key=lambda x: (x.priority, x.location))
50
51         lines = []
52         for i, issue in enumerate(self.issues, 1):
53             lines.append(f"❑ Ошибка #{i}")
54             lines.append(f"❑ Раздел: {issue.rule}")
55             lines.append(f"❑ Где: {issue.location}")
56             lines.append(f"❑ Что нужно исправить: {issue.message}")
57             lines.append("")
58         return "\n".join(lines)
59
60 class WordMethodicalChecker:
61     def __init__(self, visible: bool = False, mark_document: bool = True):
62         if dynamic is None:
63             raise RuntimeError(f"pywin32 не установлен: {WIN32_IMPORT_ERROR}")
64         self.word = None
65         self.visible = visible
66         self.mark_document = mark_document
67
68     def __enter__(self):
69         try:

```

```

70         self.word = dynamic.Dispatch("Word.Application")
71
72         self.word.DisplayAlerts = 0
73
74         try:
75             self.word.Visible = self.visible
76         except Exception as e:
77             print(f"Warning: Не удалось установить Word.Visible: {e}")
78
79         except Exception as e:
80             raise RuntimeError(f"Не удалось запустить Microsoft Word через COM: {e}")
81
82         return self
83
84     def __exit__(self, exc_type, exc_val, exc_tb):
85         if self.word:
86             self.word.Quit(False)
87
88     def open_document(self, path, readonly=True):
89         return self.word.Documents.Open(os.path.abspath(path), ReadOnly=
            readonly)
90
91     def check(self, path):
92         report = CheckReport()
93
94         checked_path = None
95         open_path = path
96         readonly = True
97
98         if self.mark_document:
99             base, ext = os.path.splitext(path)
100             checked_path = base + "_checked" + ext
101             shutil.copy(path, checked_path)
102             open_path = checked_path
103             readonly = False
104
105         doc = self.open_document(open_path, readonly=readonly)
106
107         try:
108             self._check_paragraphs(doc, report)
109             self._check_margins(doc, report)
110             self._check_page_numbers(doc, report)
111
112             if self.mark_document:
113                 doc.Save()
114         finally:
115             doc.Close(False)
116
117         return report, checked_path
118
119
120     def _mark_range(self, doc, start_par, end_par, message):
121         if not self.mark_document:

```

```

122         return
123
124     try:
125         start = doc.Paragraphs(start_par).Range.Start
126         end = doc.Paragraphs(end_par).Range.End
127         rng = doc.Range(start, end)
128
129         rng.Font.Color = WD_COLOR_RED
130         doc.Comments.Add(rng, message)
131     except Exception:
132         pass
133
134     def _mark_page(self, doc, page, message):
135         if not self.mark_document:
136             return
137
138         try:
139             start = doc.GoTo(1, 1, page).Start
140             try:
141                 end = doc.GoTo(1, 1, page + 1).Start
142             except Exception:
143                 end = doc.Content.End
144
145             rng = doc.Range(start, end)
146             rng.Font.Color = WD_COLOR_RED
147             doc.Comments.Add(rng, message)
148         except Exception:
149             pass
150
151     def _group_pages(self, doc, report, pages, message, rule="3", priority=1)
152     :
153         pages = sorted(set(pages))
154         if not pages:
155             return
156
157         start = prev = pages[0]
158
159         for p in pages[1:] + [None]:
160             if p != prev + 1:
161                 loc = f"Стр. {start} - Стр. {prev}"
162
163                 report.issues.append(
164                     CheckIssue(
165                         rule=rule,
166                         severity="ERROR",
167                         location=loc,
168                         message=message,
169                         priority=priority,
170                     )
171                 )
172
173                 for pg in range(start, prev + 1):
174                     self._mark_page(doc, pg, message)

```

```

175         if p is not None:
176             start = p
177         prev = p
178
179     def _check_paragraphs(self, doc, report):
180         font_pages = []
181         size_pages = []
182         indent_pages = []
183         spacing_pages = []
184
185         for i in range(1, doc.Paragraphs.Count + 1):
186             p = doc.Paragraphs(i)
187             text = (p.Range.Text or "").strip()
188
189             if not text or len(text) < 3:
190                 continue
191
192             try:
193                 if p.Range.Information(WD_WITHIN_TABLE):
194                     continue
195             except Exception:
196                 pass
197
198             try:
199                 style_name = str(p.Range.Style.NameLocal).lower()
200                 if "heading" in style_name or "заголол" in style_name:
201                     continue
202             except Exception:
203                 pass
204
205             page = p.Range.Information(WD_ACTIVE_END_PAGE_NUMBER)
206
207             indent = p.Range.ParagraphFormat.FirstLineIndent
208             if abs(indent) < 0.01:
209                 try:
210                     indent = p.Range.Style.ParagraphFormat.FirstLineIndent
211                 except Exception:
212                     pass
213
214             indent_cm = pt_to_cm(indent)
215             if abs(indent_cm - 1.25) > 0.05:
216                 indent_pages.append(page)
217
218             font = str(p.Range.Font.Name).strip()
219             if font != "Times New Roman":
220                 font_pages.append(page)
221
222             try:
223                 size = float(p.Range.Font.Size)
224             except Exception:
225                 size = 16.0
226
227             if size == 9999999.0:
228                 real_bad_size_found = False

```

```

229
230         try:
231             words = p.Range.Words
232             for j in range(1, words.Count + 1):
233                 w = words(j)
234                 w_text = (w.Text or "").strip()
235
236                 if not w_text:
237                     continue
238
239                 if w_text in {".", "/", ";", ":", "-", "—", "(",
240                               , ")", "[", "]", "{", "}", "/", "\\"}:
241                     continue
242
243                 try:
244                     w_size = float(w.Font.Size)
245                 except Exception:
246                     continue
247
248                 if w_size == 9999999.0:
249                     continue
250
251                 if abs(w_size - 16) > 0.2:
252                     real_bad_size_found = True
253                     break
254
255                 if real_bad_size_found:
256                     size_pages.append(page)
257
258             except Exception:
259                 pass
260         else:
261             if abs(size - 16) > 0.2:
262                 size_pages.append(page)
263
264         spacing = int(p.Range.ParagraphFormat.LineSpacingRule)
265         if spacing != WD_LINE_SPACE_SINGLE:
266             spacing_pages.append(page)
267
268     self._group_pages(
269         doc,
270         report,
271         font_pages,
272         (
273             "В основном тексте используется неверный шрифт. "
274             "Необходимо установить шрифт Times New Roman для всего
275             основного текста документа."
276         ),
277         rule="3",
278         priority=1,
279     )
280
281     self._group_pages(
282         doc,

```



```

281         report,
282         size_pages,
283         (
284             "В основном тексте используется неверный размер шрифта. "
285             "Необходимо установить размер шрифта 16 pt для всего
                основного текста документа."
286         ),
287         rule="3",
288         priority=1,
289     )
290
291     self._group_pages(
292         doc,
293         report,
294         indent_pages,
295         (
296             "Обнаружен неверный абзацный отступ первой строки. "
297             "Необходимо установить отступ первой строки 1,25 см."
298         ),
299         rule="3",
300         priority=1,
301     )
302
303     self._group_pages(
304         doc,
305         report,
306         spacing_pages,
307         (
308             "Обнаружен неверный межстрочный интервал. "
309             "Необходимо установить одинарный межстрочный интервал для
                основного текста документа."
310         ),
311         rule="3",
312         priority=1,
313     )
314
315     def _check_margins(self, doc, report):
316         pages = []
317
318         for i in range(1, doc.Sections.Count + 1):
319             sec = doc.Sections(i)
320             ps = sec.PageSetup
321
322             if (
323                 abs(pt_to_cm(ps.TopMargin) - 3.0) > 0.05 or
324                 abs(pt_to_cm(ps.BottomMargin) - 2.25) > 0.05 or
325                 abs(pt_to_cm(ps.LeftMargin) - 2.2) > 0.05 or
326                 abs(pt_to_cm(ps.RightMargin) - 2.3) > 0.05
327             ):
328                 pages.append(i)
329
330     self._group_pages(
331         doc,
332         report,

```

```

333         pages,
334         (
335             "Обнаружены неверные размеры полей страницы. "
336             "Необходимо установить поля документа: верхнее — 3 см, "
337             "нижнее — 2,25 см, левое — 2,2 см, правое — 2,3 см."
338         ),
339         rule="3",
340         priority=1,
341     )
342
343     def _check_page_numbers(self, doc, report):
344         total = doc.ComputeStatistics(WD_STATISTIC_PAGES)
345
346         found_top = False
347         found_bottom = False
348         issues = []
349
350         for i in range(1, doc.Sections.Count + 1):
351             sec = doc.Sections(i)
352
353             header = sec.Headers(WD_HEADER_FOOTER_PRIMARY)
354             footer = sec.Footers(WD_HEADER_FOOTER_PRIMARY)
355
356             for f in header.Range.Fields:
357                 if f.Type == WD_FIELD_PAGE:
358                     found_top = True
359                     try:
360                         para = f.Result.Paragraphs(1)
361                         if int(para.Alignment) != WD_ALIGN_CENTER:
362                             issues.append(
363                                 "Номер страницы расположен в верхнем
364                                 колонтитуле, но выровнен не по центру. "
365                                 "Необходимо выровнять номер страницы строго
366                                 по центру."
367                             )
368                     except Exception:
369                         issues.append(
370                             "Не удалось определить выравнивание номера
371                             страницы. "
372                             "Проверьте, что номер страницы расположен сверху
373                             и строго по центру."
374                         )
375
376             if footer.PageNumbers.Count > 0:
377                 found_bottom = True
378
379         loc = f"Стр. 1 - Стр. {total}"
380
381         if not found_top and not found_bottom:
382             message = (
383                 "Нумерация страниц отсутствует. "
384                 "Необходимо добавить номер страницы в верхней части страницы
385                 и выровнять его по центру."
386             )

```

```

382         report.issues.append(CheckIssue("4", "ERROR", loc, message, 1))
383         self._mark_page(doc, 1, message)
384         return
385
386     if not found_top and found_bottom:
387         message = (
388             "Номер страницы расположен внизу страницы. "
389             "По требованиям номер страницы должен находиться вверху
390             страницы и быть выровнен по центру."
391         )
392         report.issues.append(CheckIssue("4", "ERROR", loc, message, 1))
393         self._mark_page(doc, 1, message)
394         return
395
396     if issues:
397         message = " ".join(issues)
398         report.issues.append(CheckIssue("4", "ERROR", loc, message, 1))
399         self._mark_page(doc, 1, message)

```

email

```

1  import smtplib
2  import ssl
3  from email.mime.multipart import MIMEMultipart
4  from email.mime.base import MIMEBase
5  from email.mime.text import MIMEText
6  from email import encoders
7  from email.header import Header
8  import os
9  from typing import List
10 from fastapi import HTTPException
11 import logging
12
13 from app.core.email_config import (
14     SMTP_HOST, SMTP_PORT, SMTP_USER,
15     SMTP_PASSWORD, SMTP_RIO_EMAIL,
16     SMTP_FROM_EMAIL, SMTP_FROM_NAME
17 )
18
19 logger = logging.getLogger(__name__)
20
21
22 def send_email_with_attachments(
23     to_email: str,
24     subject: str,
25     body: str,
26     file_paths: List[str],
27     file_names: List[str] = None
28 ) -> bool:
29     """
30     Отправка email с вложениями
31     """
32     if not SMTP_USER or not SMTP_PASSWORD:
33         raise HTTPException(
34             status_code=500,

```

```

35         detail="SMTP настройки не заданы. Проверьте .env файл"
36     )
37
38     try:
39         msg = MIMEMultipart()
40         msg['From'] = SMTP_USER
41         msg['To'] = to_email
42         msg['Subject'] = Header(subject, 'utf-8').encode()
43
44         full_body = f"От: {SMTP_FROM_NAME} <{SMTP_FROM_EMAIL}>\n\n{body}"
45         msg.attach(MIMEText(full_body, 'plain', 'utf-8'))
46
47         for idx, file_path in enumerate(file_paths):
48             if not os.path.exists(file_path):
49                 logger.warning(f"Файл не найден: {file_path}")
50                 continue
51
52             with open(file_path, "rb") as attachment:
53                 part = MIMEBase('application', 'octet-stream')
54                 part.set_payload(attachment.read())
55                 encoders.encode_base64(part)
56
57                 if file_names and idx < len(file_names):
58                     filename = file_names[idx]
59                 else:
60                     filename = os.path.basename(file_path)
61
62                 part.add_header(
63                     'Content-Disposition',
64                     'attachment',
65                     filename=Header(filename, 'utf-8').encode()
66                 )
67                 msg.attach(part)
68
69         if SMTP_PORT == 465:
70             context = ssl.create_default_context()
71             with smtplib.SMTP_SSL(SMTP_HOST, SMTP_PORT, context=context) as server:
72                 server.login(SMTP_USER, SMTP_PASSWORD)
73                 server.send_message(msg)
74         else:
75             with smtplib.SMTP(SMTP_HOST, SMTP_PORT) as server:
76                 server.starttls()
77                 server.login(SMTP_USER, SMTP_PASSWORD)
78                 server.send_message(msg)
79
80         logger.info(f"Письмо успешно отправлено на {to_email}")
81         return True
82
83     except smtplib.SMTPAuthenticationError as e:
84         logger.error(f"Ошибка авторизации: {e}")
85         raise HTTPException(
86             status_code=500,

```

```

87         detail=f"Ошибка авторизации. Проверьте пароль приложения. {str(e)}
            }"
88     )
89     except Exception as e:
90         logger.error(f"Ошибка: {e}")
91         raise HTTPException(status_code=500, detail=f"Ошибка отправки: {str(e)}")
92
93
94     def send_to_rio(
95         user_fio: str,
96         user_email: str,
97         comment: str,
98         file_paths: List[str],
99         file_names: List[str]
100     ) -> bool:
101         """
102         Отправка файлов в RIO
103         """
104         subject = f"Новые методические материалы от {user_fio}"
105
106         body = f"""
107         Уважаемые сотрудники RIO!
108
109         Пользователь {user_fio} ({user_email}) отправил на проверку следующие файлы:
110
111         {chr(10).join(f'- {name}' for name in file_names)}
112
113         Комментарий отправителя:
114         {comment if comment else 'Без комментария'}
115
116         ---
117         Это автоматическое сообщение, пожалуйста, не отвечайте на него.
118
119         Сгенерировано системой проверки файлов.
120         """
121
122         return send_email_with_attachments(
123             to_email=SMTP_RIO_EMAIL,
124             subject=subject,
125             body=body,
126             file_paths=file_paths,
127             file_names=file_names
128         )

```

authService

```

1 from datetime import datetime, timezone
2
3 from fastapi import HTTPException, Request, status
4 from sqlalchemy.orm import Session
5
6 from app.core.security import (
7     create_access_token,
8     create_refresh_token,

```

```

9     get_refresh_token_expiry,
10    get_refresh_token_hash,
11    hash_password,
12    verify_password,
13 )
14 from app.models.department import Department
15 from app.models.faculty import Faculty
16 from app.models.refresh_token import RefreshToken
17 from app.models.user import User
18
19 ALLOWED_ROLES = {
20     "Доцент",
21     "Профессор",
22     "Старший преподаватель",
23     "Преподаватель",
24     "Аспирант",
25 }
26
27
28 def register_user(db: Session, fio: str, email: str, password: str,
29                 faculty_code: str, department_name: str, role: str) -> User:
30     existing_user = db.query(User).filter(User.email == email).first()
31     if existing_user:
32         raise HTTPException(status_code=400, detail="Пользователь с таким
33                             email уже существует")
34
35     if role not in ALLOWED_ROLES:
36         raise HTTPException(status_code=400, detail="Недопустимая роль")
37
38     faculty = db.query(Faculty).filter(Faculty.faculty_code == faculty_code).
39         first()
40     if not faculty:
41         raise HTTPException(status_code=404, detail="Факультет с таким кодом
42                             не найден")
43
44     department = db.query(Department).filter(Department.department_name ==
45         department_name).first()
46     if not department:
47         raise HTTPException(status_code=404, detail="Кафедра с таким
48                             названием не найдена")
49
50     if department.id_faculty != faculty.id_faculty:
51         raise HTTPException(status_code=400, detail="Кафедра не принадлежит
52                             указанному факультету")
53
54     user = User(
55         fio=fio,
56         email=email,
57         password_hash=hash_password(password),
58         role=role,
59         id_faculty=faculty.id_faculty,
60         id_department=department.id_department,
61     )

```

```

56     db.add(user)
57     db.commit()
58     db.refresh(user)
59     return user
60
61
62 def login_user(db: Session, email: str, password: str, request: Request) ->
tuple[str, str]:
63     user = db.query(User).filter(User.email == email).first()
64     if not user or not verify_password(password, user.password_hash):
65         raise HTTPException(
66             status_code=status.HTTP_401_UNAUTHORIZED,
67             detail="Неверный email или пароль"
68         )
69
70     access_token = create_access_token(user_id=user.id_user, email=user.email
)
71     refresh_token = create_refresh_token()
72
73     refresh_record = RefreshToken(
74         token_hash=get_refresh_token_hash(refresh_token),
75         expires_at=get_refresh_token_expiry(),
76         revoked=False,
77         user_agent=request.headers.get("user-agent"),
78         ip_address=request.client.host if request.client else None,
79         id_user=user.id_user,
80     )
81
82     db.add(refresh_record)
83     db.commit()
84
85     return access_token, refresh_token
86
87
88 def refresh_user_token(db: Session, refresh_token: str) -> str:
89     token_hash = get_refresh_token_hash(refresh_token)
90
91     token_record = (
92         db.query(RefreshToken)
93         .filter(RefreshToken.token_hash == token_hash)
94         .first()
95     )
96
97     if not token_record:
98         raise HTTPException(status_code=401, detail="Refresh token не найден")
99
100     if token_record.revoked:
101         raise HTTPException(status_code=401, detail="Refresh token отозван")
102
103     expires_at = token_record.expires_at
104     if expires_at.tzinfo is None:
105         expires_at = expires_at.replace(tzinfo=timezone.utc)
106

```

```

107     if expires_at < datetime.now(timezone.utc):
108         raise HTTPException(status_code=401, detail="Refresh token истёк")
109
110     user = db.query(User).filter(User.id_user == token_record.id_user).first
111         ()
112     if not user:
113         raise HTTPException(status_code=401, detail="Пользователь не найден")
114
115     return create_access_token(user_id=user.id_user, email=user.email)
116
117 def logout_user(db: Session, refresh_token: str):
118     token_hash = get_refresh_token_hash(refresh_token)
119
120     token_record = (
121         db.query(RefreshToken)
122         .filter(RefreshToken.token_hash == token_hash)
123         .first()
124     )
125
126     if not token_record:
127         raise HTTPException(status_code=404, detail="Refresh token не найден")
128
129     token_record.revoked = True
130     db.commit()
131
132
133 def logout_all_user_sessions(db: Session, user_id: int):
134     tokens = db.query(RefreshToken).filter(
135         RefreshToken.id_user == user_id,
136         RefreshToken.revoked == False
137     ).all()
138
139     for token in tokens:
140         token.revoked = True
141
142     db.commit()

```


Место для диска